

SOFTWARE TESTING

From Beginner to SDET

Manual testing, Python, pytest,
Playwright and API automation

01-05

Think and explore

Requirements, risk, manual testing

06-13

Program with Python

Values, models, files, concurrency

14-18

Prove backend behavior

Pytest, fixtures, fakes, state

19-28

Automate the browser

Playwright, controls, traces, mobile

29-34

Connect the layers

Frameworks, APIs, identity, security

35-40

Deliver as an SDET

CI/CD, resilience, capstone, review

40 chapters | Worked examples | Exercises | Executable lab

Built around your trading application, with a separate paper-only practice environment.

Learning edition 1 | 24 September 2026

Contents

How to use this book	4
Part I. Learn to Think Like a Tester	5
Chapter 1. The Application, the Learner and the Safety Boundary	5
Chapter 2. Testing Foundations, SDLC and STLC	6
Chapter 3. Requirements, Test Plans and Traceability	7
Chapter 4. Test Design Techniques with Worked Tables	9
Chapter 5. Manual Testing, Exploration, Accessibility and Defects	10
Part II. Python for Test Engineers	12
Chapter 6. Environment, Interpreter and Your First Program	12
Chapter 7. Values, Operators, Strings and Formatting	13
Chapter 8. Conditions, Loops and Collections	14
Chapter 9. Functions, Scope, Modules and Packages	15
Chapter 10. OOP, Inheritance, Polymorphism and Abstraction	16
Chapter 11. Files, JSON, CSV and Excel Test Data	18
Chapter 12. Exceptions, Logging and Debugging	19
Chapter 13. Advanced Python for Reliable Automation	20
Part III. Pytest and Backend Test Engineering	22
Chapter 14. Your First Pytest Tests and Assertions	22
Chapter 15. Fixtures, Scope and Safe Cleanup	23
Chapter 16. Parameterization, Markers, Coverage and Async Tests	24
Chapter 17. Mocks, Fakes, Spies and Broker Contract Tests	26
Chapter 18. Database, Redis, Transactions and Concurrency Tests	27
Part IV. Playwright Browser Automation	29
Chapter 19. The Web, DOM, Browser and First End-to-End Test	29
Chapter 20. CSS, XPath, Shadow DOM and Selector Tools	30
Chapter 21. Inputs, Radios, Checkboxes and Dropdowns	31
Chapter 22. Static Tables, Dynamic Rows and Pagination	32
Chapter 23. Date Pickers, Dialogs and Frames	33
Chapter 24. Mouse, Keyboard, Uploads and Downloads	34
Chapter 25. Browser Contexts, Tabs and Authentication State	35
Chapter 26. Waiting, Screenshots, Video, Traces and Flaky Tests	36
Chapter 27. Network Mocking and WebSocket Evidence	37
Chapter 28. Mobile, Cross-Browser and Parallel Testing	39

Part V. Framework Design and API Automation	40
Chapter 29. Page Object Model and Framework Design, Part One	40
Chapter 30. Framework Design, Parts Two and Three: Data, Reports and Maintenance	41
Chapter 31. HTTP, REST and CRUD API Testing	42
Chapter 32. Headers, Cookies and JSON Schema Validation	43
Chapter 33. Authentication, Email Verification, TOTP and Authorization	44
Chapter 34. Frontend-to-Backend Integration and Security Testing	46
Part VI. Delivery, Reliability and SDET Practice	48
Chapter 35. Git, GitHub and Reviewable Test Changes	48
Chapter 36. GitHub Actions and Jenkins Pipelines	49
Chapter 37. Codegen, Playwright MCP, PyCharm and Copilot	50
Chapter 38. Performance, Resilience and Observability	51
Chapter 39. Full-Project Feature Matrix and Capstone	53
Chapter 40. From Beginner to SDET: Study Plan and Interview Practice	54
Part VII. Workbook and Reference	57
Appendix A. Manual Test Pack You Can Execute and Extend	57
Appendix B. Worked Solutions and Debugging Drills	60
Appendix C. Syllabus Coverage and Additional Depth	62
Appendix D. Glossary and Release Report	64
Appendix E. Primary Sources and Reading Order	66
Appendix F. Executable Lab Source and Walkthrough	67

How to use this book

An original practical textbook using an alert-driven trading application.

This edition covers manual testing, Python, pytest, Playwright, HTTP APIs, authentication, test frameworks, CI/CD and reliability. It is written for a beginner who wants to understand and build the tests, not merely run generated scripts. The supplied course outline is a coverage checklist, not source text.

Read chapters in order. Before executing an example, predict its result. Afterward, change one input and explain the new behavior. Keep a learning journal with the requirement, oracle, test, failure and conclusion. Each chapter ends with practice and a checkpoint; Appendix B gives detailed debugging drills.

Safety boundary: all executable exercises use a local paper-only lab with synthetic symbols. Never use a client VM, real webhook URL, live broker account, production Redis or real credentials. No test in this book authorizes a trade. The lab intentionally omits production authentication, persistence and broker execution. Never expose it publicly.

Examples marked Recipe illustrate a technique and require the stated fixture, HTML or interface. The companion files are the executable reference. PDF lines may wrap for printing; use the editable source for exact indentation and long commands. Windows and Linux commands are labelled where they differ.

Forty chapters and a capstone provide a substantial learning path, not a guarantee of employment, certification, universal security or bug-free software. Record the actual scope of every test run. See VERIFICATION.md for this edition's executable checks and their limitations.

Part I. Learn to Think Like a Tester

Chapter 1. The Application, the Learner and the Safety Boundary

What you are learning

A tester turns a claim into an experiment. A developer says, "A target hit closes the position." The tester asks: Which price? Which timestamp? Which position? Does a sell request mean the sell filled? What happens if the response is lost? This book starts with those questions before teaching tools. Tools repeat instructions quickly; they do not choose the correct expectation for you.

Our case study is an alert-driven trading application. An administrator configures a named strategy. An external scanner sends candidate symbols. The application checks configuration, time, direction, sizing and optional sector/breakout rules. An execution component contacts a broker. Reconciliation compares the application's records with broker evidence. A dashboard displays positions and P&L. Market-data and browser WebSockets are different connections: the dashboard can receive app messages even when broker prices are stale.

Draw the important boundaries before writing tests:

TEXT

```

Scanner -> HTTP intake -> durable work -> decision -> broker adapter
                                   |           |
                                   configuration order evidence
                                   |           |
Browser <- API / dashboard feed <- positions <- reconciliation
                                   ^
Broker market feed -> normalized ticks -> risk decisions

```

Each arrow can lose, delay, duplicate or corrupt information. "The process is running" tests none of those arrows. "Subscription sent" is not the same observation as "a fresh tick was consumed for this symbol." Good test names describe the arrow and the guarantee being tested.

Three kinds of examples

The **case study** refers to actual project modules such as `app/main.py`, `app/trade_engine.py`, `app/dhan_broker.py`, `app/redis_store.py` and `app/services/breakout_monitor.py`. Explore them read-only first. The **lab** is the small companion application in `lab/`; it cannot contact a broker. A **recipe** is a focused example to adapt after providing the stated HTML, fixture or interface. A recipe is not a promise that the existing dashboard contains that exact label or endpoint.

You do not need market experience. A BUY position gains before costs when price rises; a SELL position gains before costs when price falls. A target is a desired exit threshold. A stop is a protective exit threshold. A tick is a price update, not proof of an executable price. A fill is an actual executed quantity. The lab assumes synthetic fills so you can study software mechanics; the production system must confirm execution.

Your first experiment

For a synthetic BUY of 2 shares at 100, target 1%, stop 0.5%: expected target is 101, stop is 99.50. A tick of 100.75 produces unrealized P&L 1.50 before costs. A tick of 101 closes the simplified lab with P&L 2.00. Write these numbers on paper before running a test. That independent calculation is your **oracle**, the source of expected results.

Keep synthetic symbols, localhost URLs and disposable data. Never run automation against a client IP. A single recorded click on "Square Off All" can become a real financial action when replayed in the wrong environment. Do not put real tokens, webhook secrets, TOTP seeds, session cookies or screenshots containing them in the book exercises or Git.

Practice and checkpoint

Draw five boundaries in the real app and write one failure for each. Explain why a dashboard WebSocket connection cannot prove broker tick freshness. A strong answer names the two independent channels and asks for a symbol-specific receive timestamp and source. A weak answer says "the green badge proves everything works."

Chapter 2. Testing Foundations, SDLC and STLC

Error, defect and failure

A human may misunderstand a requirement: that is an error. Code may use `value` or `default`, accidentally replacing a configured numeric zero: that is a defect. The user saves zero retries but later sees one retry: that is an observable failure. Debugging locates and repairs causes. Testing designs and executes observations that reveal failures or increase confidence. Quality assurance also improves the process that produces the software; it is broader than executing test cases.

Verification asks whether a work product follows its specification. Validation asks whether the result serves the user's need. Reviewing a formula is verification. Having a trader confirm that an exit alert closes only positions owned by the matching strategy contributes to validation. Both can be done early; neither is restricted to a final stage.

Development and test lifecycles

The software development lifecycle includes discovery, requirements, design, implementation, verification, release and maintenance. Iterative teams revisit these stages rather than completing each once. The software testing lifecycle includes analyzing requirements, planning, designing tests, preparing data/environment, executing, reporting defects, retesting, regression testing and reporting residual risk. Testing should begin during requirements discussions: an ambiguous rule is much cheaper to fix before it becomes five services and a dashboard.

Consider capital 10,000 and stock price 12,000. Should quantity be zero or one? Both are implementable, but only one matches a chosen business policy. A "minimum one" policy intentionally exceeds the per-trade capital value. A test cannot resolve the business disagreement; it must record the chosen policy and its consequences. Never hide such a decision inside a test helper.

Levels and purposes

Test level	Example	What remains unproved
Unit	BUY target calculation	HTTP parsing and broker fills
Integration	API saves configuration and store reads it	Real browser interaction
System	Browser, API, workers and disposable Redis	Live brokerage availability
Acceptance	User validates strategy-owned exit workflow	All possible failure sequences

Functional testing checks behavior. Nonfunctional testing checks characteristics such as latency, resilience, accessibility, security and maintainability. Smoke testing asks whether a build is usable enough for deeper testing. Regression testing checks whether changes broke previously working behavior. Retesting executes the specific failed scenario after its fix. Exploratory testing combines learning, test design and execution in a bounded session.

Why complete testing is impossible

Ten binary toggles already have 1,024 combinations before symbols, prices, time, broker responses and restarts are considered. Testing therefore selects risks deliberately. Prioritize duplicated orders, unauthorized actions and wrong exit quantities above cosmetic spacing. Test concentration is not negligence when the risk argument and remaining gaps are explicit.

Do not equate coverage with confidence. A line can execute while its result is never checked. A browser can click every button without verifying the persisted values. A suite of 300 tests can miss one damaging interleaving. Your report should say what you observed, under which conditions, and what you did not exercise.

Practice and checkpoint

Classify "wrong retry count after reload," "response takes 20 seconds," and "another user can view credentials." They are respectively functional persistence, performance and authorization concerns, though categories overlap. Write one unit and one integration test idea for the zero-retry defect. The integration test must save zero, reload from the backend and compare the returned value, not merely read the input immediately after typing.

Chapter 3. Requirements, Test Plans and Traceability

Make a requirement observable

"Entries must be fast" is an aspiration. A testable statement defines measurement points, load, environment and an acceptance threshold. For example: "Under a synthetic burst of 20 alerts per second on the specified staging machine, 95% of valid webhook acknowledgments finish within the agreed budget; none wait for broker execution." The budget is a stakeholder decision. Do not advertise a made-up latency as an existing app guarantee.

Use Given/When/Then to clarify behavior:

GHERKIN

```

Given strategy alpha owns an OPEN TEST position
And strategy beta owns another OPEN TEST position
When alpha's configured exit alert includes TEST
Then alpha's remaining quantity is submitted for exit once
And beta's position is unchanged
And alpha is CLOSED only after confirmed full exit fills

```

The final sentence prevents a dangerous shortcut: equating a submitted exit request with closure. Add examples for partial fills, rejected exits and duplicate callbacks. Acceptance criteria should distinguish intentional behavior from missing implementation.

A compact test plan

State scope, risks, environments, data, responsibilities, schedule, entry criteria, exit criteria and reporting. For our release, scope includes configuration persistence, webhook-to-watch flow, execution idempotency and position display. Exclusions include actual exchange matching and broker uptime. The environment uses a fake broker and disposable Redis. Entry criteria include an identified commit and installed dependencies. Exit criteria include all critical safety scenarios passing, unresolved high-risk defects reviewed, and evidence attached. A deadline alone is not an exit criterion.

Create a traceability matrix before implementation grows:

Requirement	Manual experiment	Automated evidence
R01 preserve numeric zero	Save and reload retries=0	API persistence plus browser reload
R02 strategy-owned exit	Exit alpha, observe beta	Two-position integration test
R03 candle TTL fixed	Delay reference candle	Injected clock and watch expiry
R04 no false LIVE row	Earlier skip, later entry	Dashboard row identity test
R05 broker acceptance is not fill	Return PENDING	Adapter and reconciliation test

A requirement can have several tests. A test can cover several requirements, but avoid hiding ten unrelated failures behind one enormous scenario. Link defect IDs and test IDs so the team can see why a regression test exists six months later.

Worked test case

ID: CFG-ZERO-01. **Precondition:** isolated app, authenticated test admin, strategy demo exists. **Data:** pending retries 0, limit buffer 0. **Steps:** open strategy; set both zero; save; confirm successful response; reload the page; reopen the same strategy. **Expected:** both fields and backend JSON remain zero. **Cleanup:** delete only this test strategy. **Evidence:** redacted response, screenshot and build identifier. **Negative variant:** omit a field entirely; its documented default should apply. Zero and missing are distinct partitions.

Store test cases as useful engineering artifacts, not ceremonial spreadsheets. A CSV is fine for a small team. Include title, requirement, setup, input, action, expected result, actual result, status, defect and evidence link. Avoid "works correctly" in the expected-result column: it gives the tester nothing concrete to compare.

Practice and checkpoint

Rewrite "websocket works" into three requirements covering transport, subscription and freshness. Expected answer: a connection is established; the required security ID is included; a validated tick with acceptable age reaches the risk consumer. Add a fourth for recovery after credentials change. None is proved by only receiving HTTP 200 from /api/broker-status.

Chapter 4. Test Design Techniques with Worked Tables

Equivalence partitioning and boundaries

Equivalence partitioning groups inputs expected to behave alike. For quantity constrained to integer 1 through 100, useful groups are valid integers, integers below range, integers above range, fractional numbers, strings, booleans, null and missing. Python's `bool` is related to `int`, so `True` is a valuable negative example when a strict integer is required.

Boundary-value analysis targets edges: 0, 1, 2, 99, 100 and 101. For entry time 09:15 through 09:50, first define whether endpoints are inclusive. Then test just before start, exactly start, just after start, just before end, exactly end and just after end. Use timezone-aware timestamps. A test using your laptop's local time silently changes meaning on a UTC server.

Decision tables

When several conditions interact, enumerate decisions explicitly. A simplified entry table is:

Config exists	Enabled	Kill switch	Expected
No	any	any	CFG_MISSING, no submission
Yes	No	Off	disabled, no submission
Yes	Yes	On	blocked, no submission
Yes	Yes	Off	evaluate remaining filters

"Any" means the result does not depend on that condition for this rule. It does not mean those combinations were all executed. Add sector readiness, fresh prices and duplicate positions in a second table rather than creating an unreadable sheet. If two rejection reasons apply, agree which takes precedence and whether reason ordering is a contract.

State-transition testing

An order is not a Boolean success flag. Model transitions such as `CREATED -> SUBMITTED -> PENDING -> PARTIAL -> COMPLETE`, and `PENDING -> CANCELLED` or `REJECTED`. Include an `UNKNOWN` outcome after a network timeout. A valid state-transition test checks both the next state and side effects. After `PARTIAL` with requested 10 and filled 4, remaining is 6. A duplicate `PARTIAL` callback must not add another 4. A late `COMPLETE` after a cancel request requires reconciliation, not blind replacement.

PYTHON

```
def remaining(requested, filled):
    if not 0 <= filled <= requested:
        raise ValueError("Inconsistent broker quantity")
    return requested - filled

assert remaining(10, 4) == 6
assert remaining(10, 10) == 0
```

The oracle is independently derived arithmetic plus the broker contract. A reported filled quantity greater than requested must not be "fixed" by silently clamping it to zero remaining; it is evidence that needs investigation.

Pairwise, invariants and error guessing

Pairwise selection covers each pair of factor values, reducing many combinations. It is useful for browser, viewport, product and direction combinations, but cannot replace targeted three-way safety scenarios. Explicitly test SHORT + sector filter + losers selection even if a generated matrix happens to omit it.

Invariants hold across many sequences: a BUY trailing stop never decreases; a SELL trailing stop never increases; filled quantity cannot be negative; a rejected entry cannot create a filled position; exit quantity cannot exceed owned remaining quantity. Error guessing uses incident experience: mismatched alert names, stale secrets, daylight/timezone assumptions, duplicate webhook delivery and SDK error dictionaries returned with successful HTTP transport.

Practice and checkpoint

Design six tests for breakout TTL=2 minutes. Include before expiry, exact expiry, after expiry, delayed candle, restart and duplicate alert. The essential answer is that retries and duplicate deliveries must not silently extend the original expiry. Define exact-expiry behavior in the specification before asserting it.

Chapter 5. Manual Testing, Exploration, Accessibility and Defects

A complete manual lab walkthrough

Start the local lab using its README. Open the page, tab through the form without a mouse, change strategy to `manual`, quantity to 2, target to 1 and stop to 0.5. Save and read the status message. Open browser Developer Tools, Network tab, and inspect the configuration response. The lab stores a strategy through `/api/strategies`; this is deliberately different from the production configuration route.

Submit a synthetic signal from the README's localhost command, replacing its strategy name with `manual`. Refresh positions. Expect one OPEN row at entry 100. Submit the same event again. Refresh: there must still be one position. Submit a tick at 101. Refresh: CLOSED, TARGET, P&L 2.00. Record what you observed at each boundary: response, stored state and screen. The lab fills instantly by design; a live trading test would also need broker fill evidence.

Exploratory testing with a charter

A charter is a mission, not random clicking: "Explore configuration save failures for 25 minutes, looking for misleading success messages and stale values." Vary empty names, duplicate saves, browser refresh, disconnected network and invalid numbers. Keep notes with timestamp, action, observation and question. Stop and isolate a reproducible failure before changing ten more settings. End with a debrief: coverage, defects, unresolved questions and next charter.

Usability and accessibility

At 390-pixel width, check whether labels remain readable, the Save button is reachable, and horizontal scrolling is contained to a table rather than the entire page. At 200% zoom, check reflow and clipped controls. Use Tab and Shift+Tab to verify focus order, visible focus and keyboard access. Open and close modals; focus should enter the dialog and return to the initiating control. Color must not be the only way to distinguish LIVE from ERROR.

An accessible name is the name assistive technology exposes for a control. A visible label beside an input is insufficient if it is not programmatically associated. Automated role locators provide useful feedback, but they do not replace a screen-reader session, contrast review or specialist accessibility audit. Do not claim compliance from a passing Playwright suite.

Write a useful defect report

Title: Saved zero pending retries reloads as one. **Environment:** commit, browser version, disposable test account, timezone. **Steps:** minimal numbered sequence. **Expected:** zero persists. **Actual:** API or reload shows one. **Impact:** unintended additional order attempt. **Evidence:** redacted request/response and screenshot. **Severity:** impact if encountered. **Priority:** scheduling decision considering exposure and business urgency. A typo may be low severity but urgent before a public demo; duplicate order placement is high severity even if hard to reproduce.

Typical defect workflow: New -> Triaged -> In progress -> Fixed -> Retest -> Closed, or Reopened if the evidence still fails. A rejected bug needs a documented reason, such as an agreed requirement, not "works on my machine." Always record whether a failure belongs to application behavior, test code, test data or the environment.

UAT and checkpoint

User acceptance testing uses representative workflows and business expectations. Ask the user to configure two strategies, show which one owns a position, and predict which exit alert is allowed to close it. Agreement must be recorded before release. Practice writing a defect for a skipped alert shown as LIVE after a later entry. A strong expected result preserves the earlier alert's status while presenting the actual open position separately.

Part II. Python for Test Engineers

Chapter 6. Environment, Interpreter and Your First Program

Build an isolated workspace

A Python interpreter executes Python instructions. A virtual environment gives a project its own package installation directory. It does not create a security sandbox or prevent network calls. Keep the learning environment separate from the trading server and its dependencies. Open a terminal in `docs/testing-book/lab`, not the production application directory, and follow the lab README.

POWERSHELL

```
py -m venv .venv
.venv\Scripts\python -m pip install -r requirements.txt
.venv\Scripts\python -m playwright install chromium
.venv\Scripts\python -m pytest -m "not browser" -q
```

On Linux or macOS use `python3` to create the environment and `.venv/bin/python` afterward. Calling the environment's interpreter directly avoids activation-policy problems on Windows. `python -m pip` connects installation to a known interpreter; a bare `pip` can belong to a different installation. Check `python --version`, `python -m pip --version` and `python -c "import sys; print(sys.executable)"` when debugging installation issues.

Create `hello.py` in your own practice directory:

PYTHON

```
symbol = "TEST"
quantity = 2
print(f"Paper symbol: {symbol}; quantity: {quantity}")
```

The first two lines bind names to objects. The final line prints a formatted string. A comment starts with `#`; use it to explain a decision, not repeat the instruction. Python uses indentation to delimit blocks. Four spaces is the conventional indentation unit; mixing tabs and spaces creates avoidable errors. A syntax error means the interpreter cannot parse the program. An exception means valid syntax reached an invalid operation at runtime.

Organize files early

Use small modules: domain calculations, API application and tests. Do not name a practice file `json.py`, `pytest.py` or `decimal.py`, because it can shadow a package with that name. Avoid hardcoded machine paths. Use `pathlib.Path` relative to the module or fixture. A module can expose functions for import and reserve command-line actions under `if __name__ == "__main__":`.

The lab requirement ranges describe supported versions, not a reproducible deployment lock. After a successful learning run, record installed versions in a lock or freeze file. Upgrade in a branch, run tests, then review behavior. Never fix a learning import error by upgrading all packages in the production virtual environment.

Practice and checkpoint

Print the interpreter path, create a script with one indentation error, observe the traceback and fix it. Explain why activating an environment is convenient but optional when its executable path is explicit. The answer is that activation changes shell lookup; it does not change what Python itself means.

Chapter 7. Values, Operators, Strings and Formatting

Types express different meanings

Integers represent counts, strings represent labels, booleans represent decisions, and None represents an absent value. A float approximates many decimal fractions. Financial calculations often benefit from Decimal constructed from a string. Decimal(0.1) preserves a float approximation; Decimal("0.1") represents the intended decimal value.

PYTHON

```
from decimal import Decimal, ROUND_HALF_UP

entry = Decimal("427.50")
target = entry * (1 + Decimal("1") / 100)
stop = entry * (1 - Decimal("0.5") / 100)
print(target) # 431.775
print(target.quantize(Decimal("0.01"), rounding=ROUND_HALF_UP))
print(stop.quantize(Decimal("0.01"), rounding=ROUND_HALF_UP))
# Display: 431.78 and 425.36
```

Display rounding and order-price tick rounding are different requirements. A two-decimal display does not prove that an order is a valid multiple of 0.05. For an aggressive BUY limit, rounding down can make execution less likely; for a SELL limit the direction reverses. Test the policy explicitly rather than sprinkling round() calls across the adapter.

Operators and truthiness

+, -, *, /, // and % perform arithmetic; ==, < and >= compare values; and, or, not combine conditions. = assigns and == compares. is checks object identity, so use value is None but do not use is to compare symbol strings.

PYTHON

```
config = {"retries": 0}
wrong = config.get("retries") or 1
correct = config.get("retries", 1)
assert wrong == 1
assert correct == 0
```

Zero, empty strings and empty containers are false in a Boolean context. That is useful for some checks but wrong when zero is a valid setting. Missing, null, zero and empty text should be separate test inputs. A bool arriving where strict quantity is expected also deserves a test.

Strings, slicing and normalization

Strings are immutable sequences. `text.strip()` returns a new value; it does not edit text in place. `upper()`, `lower()`, `split()`, `join()` and slicing help normalize test data. Normalize according to a contract: collapsing every punctuation mark can make distinct alert names collide.

PYTHON

```
raw = " TEST, DEMO ,TEST "
symbols = list(dict.fromkeys(s.strip().upper() for s in raw.split(",")))
assert symbols == ["TEST", "DEMO"]
assert "CONFIG"[0:3] == "CON"
```

An f-string inserts values into readable messages. Use descriptive assertion messages, but never include secrets. Format P&L separately from business arithmetic. `f"{float_value:.2f}"` is a display decision, not proof that the underlying number is exact.

Practice and checkpoint

Calculate a SELL target and stop at entry 100, target 2%, stop 1%. Expected: target 98, stop 101. Explain why `bool("false")` is true: it is a nonempty string. APIs should validate booleans rather than trust arbitrary truthiness.

Chapter 8. Conditions, Loops and Collections

Follow the decision path

An `if` selects a branch; `elif` checks later alternatives; `else` handles the remainder. Order matters when multiple conditions are true. In an entry decision, a missing configuration may take precedence over a sector error. Write down this precedence rather than relying on the incidental order of branches.

PYTHON

```
def entry_reason(config, kill, already_open):
    if config is None:
        return "CFG_MISSING"
    if not config["enabled"]:
        return "DISABLED"
    if kill:
        return "KILL_SWITCH"
    if already_open:
        return "ALREADY_OPEN"
    return "EVALUATE"
```

Test each return path and at least one overlapping condition. This function is deliberately a simplified decision exercise, not the full production gate. It gives a beginner a small behavior they can understand completely before working with queues and brokers.

Lists, tuples, sets and dictionaries

A list preserves order and permits duplicates; use it for an ordered stream of events. A tuple represents a fixed grouping, such as `(strategy, symbol)`. A set represents unique membership; use it for subscribed symbols, but do not assume display order. A dictionary maps keys to values; use a composite key to prevent one strategy's position from being confused with another's.

PYTHON

```
positions = {
    ("alpha", "TEST"): {"qty": 2},
    ("beta", "TEST"): {"qty": 5},
}
assert positions[("alpha", "TEST")]["qty"] == 2
subscribed = {"TEST", "DEMO", "TEST"}
assert len(subscribed) == 2
```

Mutability creates test leaks. Assigning `other = config` creates another reference to the same dictionary. `config.copy()` is shallow: nested lists are still shared. Use fixture factories or a deliberate deep copy for nested mutable test data. Do not mutate a dictionary while iterating over its keys without planning how iteration remains valid.

Iteration and comprehensions

Use `for` to process a finite collection, `enumerate` when you need indices, and `zip` when two sequences have a meaningful pairing. `zip` normally stops at the shortest sequence, which is dangerous for malformed OHLC arrays. Validate equal lengths or use strict pairing where supported.

PYTHON

```
prices = [100, 101, 102]
quantities = [2, 3, 4]
notionals = [price * qty for price, qty in zip(prices, quantities, strict=True)]
assert notionals == [200, 303, 408]
```

A while loop needs a termination condition. Polling a broker forever can block a worker. Tests should cover a successful condition, a timeout, cancellation and a malformed response. Comprehensions are concise but should not hide side effects or complex state transitions. Prefer a readable loop when every iteration needs logging and recovery.

Practice and checkpoint

Given sector percentages `AUTO=1.5`, `IT=-2.0`, `METAL=-0.5`, select two LONG and two SHORT candidates. Expected LONG order: AUTO then METAL; SHORT: IT then METAL. This is rank ordering; whether positive-only or negative-only membership is additionally required must be a separate business rule.

Chapter 9. Functions, Scope, Modules and Packages

A function is a testable contract

A function receives inputs, performs a responsibility, and returns a result or raises an exception. Prefer explicit inputs to reading global state. `calculate_pnl(entry, ltp, qty, side)` is easier to test than a function that reads the current broker session, global position and wall clock. A pure function has no observable side effects and produces the same output for the same input.

PYTHON

```
from decimal import Decimal

def pnl(entry: Decimal, ltp: Decimal, qty: int, side: str) -> Decimal:
    if side not in {"BUY", "SELL"} or qty <= 0:
        raise ValueError("Invalid position")
    direction = 1 if side == "BUY" else -1
    return (ltp - entry) * qty * direction

assert pnl(Decimal("100"), Decimal("101"), 2, "BUY") == 2
```

Type hints document expectations and help static tools, but Python does not automatically enforce them at runtime. Validate untrusted boundaries. Avoid mutable default arguments such as `def collect(items=[]):`; the same list is reused between calls. Use `None` and construct a fresh list inside the function.

Scope and dependencies

Names can be local to a function, enclosed in an outer function, global to a module or built-in. `global` and `nonlocal` change assignment behavior, but frequent global mutation makes tests harder to isolate. Pass dependencies instead: a store, broker adapter, clock or notifier. A closure can capture a fake clock for deterministic expiry tests.

PYTHON

```
def is_expired(expires_at, now):
    return now >= expires_at

assert not is_expired(120, 119.9)
assert is_expired(120, 120)
```

The function does not sleep. It tests the rule, while another integration test verifies scheduling calls the rule. Separating policy from scheduling makes failures easier to interpret.

Module layout

The lab has `domain.py`, `server.py` and `tests/`. The server imports the model; tests import both layers as needed. Larger packages may have `domain/`, `adapters/`, `services/` and `tests/`. Avoid circular imports by depending on small interfaces and keeping startup wiring outside domain code. Importing a module should not place orders, start services or modify databases.

Use `__init__.py` when intentionally creating a regular package. Run tests from a documented directory so imports are reproducible. Do not repeatedly modify `sys.path` throughout a framework to compensate for an unclear package design. In this companion lab, running `python -m pytest` from the lab directory makes its modules importable.

Practice and checkpoint

Refactor a function that reads `datetime.now()` into one accepting `now`. Write tests before, at and after expiry. The key insight is that dependency injection is not a large framework: passing one argument can make a hidden dependency visible.

Chapter 10. OOP, Inheritance, Polymorphism and Abstraction

State and behavior together

A class defines a kind of object. An instance contains its own state. A position has identity, quantities and prices; methods such as `mark()` update it according to rules. Encapsulation means changes go through meaningful operations rather than arbitrary mutations from every caller. A leading underscore is a convention for nonpublic attributes, not a security boundary.

The lab's `Position` dataclass creates initialization and representation methods. `from_fill()` is a class method because it constructs an instance from a meaningful event. `pn1` is a property computed from current state. Read `lab/domain.py` and trace which inputs affect each field. The lab deliberately has fewer rules than the real engine.

Program to a small interface

PYTHON

```
from typing import Protocol

class Broker(Protocol):
    def order_status(self, order_id: str) -> dict: ...

class PaperBroker:
    def order_status(self, order_id: str) -> dict:
        return {"id": order_id, "status": "PENDING", "filled": 0}

def confirmed_quantity(broker: Broker, order_id: str) -> int:
    return broker.order_status(order_id)["filled"]

assert confirmed_quantity(PaperBroker(), "paper-1") == 0
```

Polymorphism means the caller works with different implementations through the same contract. A production adapter and a fake can both expose `order_status`. The fake above is intentionally minimal; a realistic contract test must include rejection, partial fill and unknown status. An abstraction is useful when it hides vendor differences without erasing essential semantics.

Inheritance represents an "is a" relationship and can reuse behavior. Composition gives an object collaborators, such as a page object receiving a Playwright Page. Prefer composition for test services and page objects unless inheritance genuinely simplifies shared behavior. A giant `BaseTest` containing login, database setup, retries and screenshots can make every test depend on hidden setup.

Test behavior, not implementation trivia

Do not assert that a private helper was called three times merely because it currently is. Assert that duplicate order updates do not increase filled quantity. Interaction assertions are useful when the interaction itself is the requirement, such as "broker submission must not occur while kill switch is on." An object-oriented design should make important boundaries inspectable, not require tests to reach into every private field.

Practice and checkpoint

Write two notifier implementations: one records messages in a list, one raises a simulated delivery failure. Inject each into a service. Explain why notification failure must not undo an already confirmed broker fill. The correct boundary separates execution state from optional notification delivery.

Chapter 11. Files, JSON, CSV and Excel Test Data

Files need a lifecycle

Use a context manager so the file closes even after an exception. Prefer explicit UTF-8 encoding for text. Relative paths depend on the working directory; fixture data should be located relative to the test module or a configured data directory. Pytest's `tmp_path` provides a disposable directory for one test, avoiding accidental edits to real files.

PYTHON

```
import json
from pathlib import Path

path = Path("cases.json")
cases = [{"side": "BUY", "entry": "100", "ltp": "101", "pnl": "2"}]
path.write_text(json.dumps(cases, indent=2), encoding="utf-8")
loaded = json.loads(path.read_text(encoding="utf-8"))
assert loaded == cases
```

JSON is a structured interchange format. It has arrays, objects, strings, numbers, booleans and null, but not Python Decimal or datetime objects. Decide an explicit serialization policy. The lab represents monetary values as strings so tests can reconstruct Decimal precisely. Do not use `eval()` to read test data; it executes code.

CSV and Excel

CSV is simple and reviewable in Git. Use `csv.DictReader`, not `line.split(",")`, because quoted fields can contain commas. Convert strings at the boundary and validate required columns. Preserve leading zeroes when a field is an identifier rather than a number.

PYTHON

```
import csv
import io

stream = io.StringIO("symbol,quantity\nTEST,2\nDEMO,3\n")
rows = list(csv.DictReader(stream))
assert int(rows[0]["quantity"]) == 2
```

Recipe, with optional `openpyxl` installed in the lab environment:

PYTHON

```
from openpyxl import load_workbook

book = load_workbook("cases.xlsx", read_only=True, data_only=True)
try:
    sheet = book["Entries"]
    for symbol, qty in sheet.iter_rows(min_row=2, values_only=True):
        assert isinstance(symbol, str)
        assert isinstance(qty, int) and qty > 0
finally:
    book.close()
```

`data_only=True` reads cached formula results; it does not calculate Excel formulas. Missing caches can produce `None`. Spreadsheet data is not automatically authoritative. Review its expected values independently, version it, and avoid duplicating the same calculation in both application and expected-data generator.

Practice and checkpoint

Create CSV cases for zero, negative and fractional quantities. Test a malformed row and a missing column. A good data loader fails with the file and row context, without echoing confidential content. Never include real customer account exports in a beginner test repository.

Chapter 12. Exceptions, Logging and Debugging

Exceptions carry meaning

An exception interrupts normal flow and travels up the call stack until handled. Catch only where you can recover, add useful context or translate to a boundary response. `except Exception: pass` makes failures disappear and can leave inconsistent trading state. A `finally` block releases resources; it does not prove the operation succeeded.

PYTHON

```
import json

def decode_quote(raw):
    try:
        data = json.loads(raw)
    except json.JSONDecodeError as exc:
        raise ValueError("Quote response was not JSON") from exc
    if "ltp" not in data:
        raise ValueError("Quote missing ltp")
    return data["ltp"]
```

Exception chaining preserves the original cause. In tests, assert both the error category and the absence of unwanted side effects. A malformed quote should not create a position. An HTML 502 page should not become a fake price of zero. A catch-all API handler can produce a safe JSON error envelope, but it must still log a redacted traceback and return an appropriate non-success status.

Debug from evidence

Read a traceback from the last exception upward. Identify the first relevant project frame, the input that reached it, and the earliest violated assumption. Distinguish root cause from consequences: a missing socket attribute during cleanup may follow failure to create an event loop because the process exhausted file descriptors. Fixing the cleanup message alone would miss the leak.

Use structured fields: service, user identifier, request ID, symbol, order correlation ID, operation and outcome. Never log access tokens, passwords, TOTP seeds or full Redis URLs. A successful response status alone is insufficient evidence of correct content. Logs should make decisions explainable without exposing credentials.

Test failures versus application failures

A strict locator matching two rows is usually a test selector problem. A timeout after clicking Save may be application latency, network failure, invalid data or a selector waiting for the wrong message. Inspect the response and trace before increasing timeout. A 50 ms performance assertion on a busy laptop can fail despite correct concurrency; coordinate workers with events to test ordering, and measure performance in a controlled benchmark.

Practice and checkpoint

Make `decode_quote` receive HTML, empty text and JSON without `ltp`. Write three exception assertions. Then write one successful case. Explain why replacing all failures with `{}` would reduce observability and may turn a broker outage into a misleading "no data" condition.

Chapter 13. Advanced Python for Reliable Automation

Time, resources and structured data

Use timezone-aware datetimes for exchange sessions and UTC for portable event timestamps. Use a monotonic clock for elapsed durations; wall-clock corrections should not extend a timeout. A dataclass gives named fields; an enum gives a controlled set of states; a generator yields values lazily. These features help express contracts, but do not eliminate validation.

PYTHON

```
from datetime import datetime
from zoneinfo import ZoneInfo

ist = ZoneInfo("Asia/Kolkata")
alert_time = datetime(2026, 9, 23, 10, 30, 3, tzinfo=ist)
assert alert_time.utcoffset().total_seconds() == 19800
```

On systems without an IANA timezone database, install `tzdata` in the lab environment. Never simulate IST by changing a UTC timestamp's label without converting the represented instant.

Async is cooperative, not magic

An `async def` function returns a coroutine. `await` allows other tasks to progress when the awaited operation cooperates. Calling a synchronous slow broker SDK directly from the event loop can still block every request. Use a bounded thread worker for blocking I/O. CPU-heavy calculations may need a process rather than more async tasks. A semaphore bounds concurrent work, but you also need admission limits when producers can outpace consumers.

PYTHON

```
import asyncio

async def collect_two(fetch):
    first, second = await asyncio.gather(fetch("TEST"), fetch("DEMO"))
    return first, second

async def paper_fetch(symbol):
    await asyncio.sleep(0)
    return {"symbol": symbol, "ltp": 100}
```

This example shows concurrent scheduling, not a speed benchmark. Shared mutable state still needs ownership rules. Test cancellation and cleanup: an interrupted task should release its local resources, but a broker request already sent may still execute remotely. Cancellation is not proof that an external order was cancelled.

Dependency injection and clocks

A deterministic test controls time, randomness and external responses. Inject a clock with `now()`, a broker with scripted results, and a store with an isolated namespace. For lease tests, advance fake time during controlled renewals; for concurrency, hold a fake market-data call on an event while checking an order worker completes. Avoid hundreds of `sleep(0.01)` calls that only work on one machine.

Decorators wrap behavior; pytest fixtures and marks use decorators. Context managers implement setup/cleanup; fixtures often use `yield` for the same lifecycle. Learn these mechanisms by writing small examples before building a large framework.

Practice and checkpoint

Build a fake clock and a breakout watch that expires at 120. Advance to 119 and 120 without sleeping. Explain why deterministic time tests do not replace a separate staging test of actual scheduling delays. One proves policy; the other observes real runtime behavior.

Part III. Pytest and Backend Test Engineering

Chapter 14. Your First Pytest Tests and Assertions

Discover, arrange, act, assert

Pytest discovers test files and functions by naming conventions. A function named `test_target_exit` in `test_domain.py` is collected; a normal helper named `make_position` is not a test. A test should arrange known inputs, act on the system, and assert an independently chosen expected result. Cleanup belongs in a fixture or a `finally` block so failures cannot prevent it.

Run this complete example from the lab directory:

PYTHON

```
from decimal import Decimal
from domain import Position

def test_buy_target_closes_position():
    position = Position.from_fill("alpha", "TEST", "BUY", 2, "100", "1", ".5")
    position.mark("101")
    assert position.status == "CLOSED"
    assert position.exit_reason == "TARGET"
    assert position.pnl == Decimal("2")
```

The first line inside the test creates the starting state. The next line simulates the observation that triggers a decision. The assertions inspect the result. Three assertions are appropriate here because they describe one behavior: closure at a target. Combining login, configuration, entry, ten unrelated validation failures and logout into one unit test makes failures difficult to diagnose.

Read a failure report

Change the expected P&L to 3 and run the test. Pytest shows the test name, failing line and comparison. The defect is now deliberately in the test, not the domain model. Undo it after observing the report. A red result means actual and expected differ; investigation determines which is wrong.

BASH

```
python -m pytest tests/test_domain.py -q
python -m pytest tests/test_domain.py -v
python -m pytest -k "target or sector" -q
python -m pytest --collect-only -q
```

`-q` reduces output, `-v` lists individual tests, `-k` filters names and `--collect-only` checks discovery without execution. A command that runs zero tests is not a passing release check. Exit codes distinguish success, test failures and collection problems; CI must preserve them rather than `append || true`.

Assert the right thing

Use exact equality for discrete values and Decimal money where the contract is exact. Use `pytest.approx` when a numerical algorithm legitimately has floating tolerance; choose a tolerance meaningful to the requirement. Do not allow a one-rupee tolerance merely to hide a paise rounding bug. Check collection contents rather than only their length when identity matters.

PYTHON

```
import pytest
from domain import quantity

def test_invalid_price_is_rejected():
    with pytest.raises(ValueError, match="positive"):
        quantity("10000", "0")
```

Checking an exception alone may be insufficient. For a stateful API, also assert that no configuration or position was written. In a broker test, assert that submission was never called. Negative outcomes often have important absence requirements.

Practice and checkpoint

Write tests for entry 100, quantity 2 and a stop at 99.50. Test 99.51, 99.50 and 99.49 separately. The expected exact-boundary behavior is closure at $\leq$ for BUY in this lab. Tests should encode this rule rather than accidentally choosing only prices far beyond the boundary.

Chapter 15. Fixtures, Scope and Safe Cleanup

A fixture supplies a dependency

A fixture is a named provider of data or resources. Pytest inspects a test's arguments and supplies matching fixtures. This is dependency injection without manual object construction in every test. `conftest.py` shares fixtures across a directory's tests without importing it explicitly.

PYTHON

```
import pytest
from fastapi.testclient import TestClient
from server import create_app

@pytest.fixture
def client():
    with TestClient(create_app()) as value:
        yield value

def test_empty_positions(client):
    response = client.get("/api/positions")
    assert response.status_code == 200
    assert response.json() == []
```

The application factory makes fresh dictionaries for this test. `yield` returns control to the test and resumes for teardown afterward. The context manager exits even if an assertion fails. Starting from empty state is not merely convenient: it prevents another test's positions from changing this test's result.

Scope is a tradeoff

Function scope creates a resource per test. Module and session scopes reuse it longer and may be faster. Reuse also increases shared-state risk. A browser process can often be session-scoped while browser contexts remain function-scoped. A mutable account reused by parallel tests can make an otherwise correct suite unreliable. Use function scope until you can explain precisely why reuse is safe.

Do not make all setup `autouse=True`. Invisible setup can connect to services even for tests that only need arithmetic. `Autouse` is appropriate for truly universal safety checks, such as refusing a production base URL. Prefer explicit dependencies for authentication, data creation and service startup.

Factories and cleanup stacks

When a test needs several strategies, return a fixture factory that records created identifiers. Teardown removes only those objects. Never "clean" a shared database with an unscoped flush. For SQL integration tests, transaction rollback may isolate data, but external workers using other connections will not automatically participate in that transaction. For Redis, unique prefixes plus bounded TTLs are safer than assuming a database number is exclusively yours.

Pytest's `tmp_path` fixture creates a separate directory for each test. Use it for uploaded CSVs, downloaded reports and JSON test data. Test-generated browser storage state contains credentials and must live in ignored, access-controlled artifacts, not a fixture committed to Git.

A fixture failure is not the same as a test failure

If the local server cannot bind or the browser executable is missing, setup errors before the behavior runs. Report that as an environment/setup problem, not proof that `Save` is broken. Conversely, do not dismiss every setup failure: a new startup crash may be the application defect the fixture reveals.

Practice and checkpoint

Create two tests: one saves a strategy, the other expects no strategies. Run them in both orders. They should pass with fresh factory state. If the second relies on the first deleting records, isolation is incomplete. Read the companion `lab_url` fixture and identify where its server is started, readiness is checked and shutdown is verified.

Chapter 16. Parameterization, Markers, Coverage and Async Tests

One rule, many representative examples

Parameterization runs the same behavior with multiple named data sets. It reduces duplicated test code but should not hide meaning in a giant spreadsheet. Prefer descriptive IDs and a few well-chosen partitions.

PYTHON

```

from decimal import Decimal
import pytest
from domain import tick_price

@pytest.mark.parametrize("side,expected", [
    pytest.param("BUY", "100.05", id="buy-rounds-up"),
    pytest.param("SELL", "100.00", id="sell-rounds-down"),
])
def test_tick_policy(side, expected):
    assert tick_price("100.03", ".05", side) == Decimal(expected)

```

The tick size is supplied test data, not a claim that every exchange instrument uses 0.05. Expand with already aligned prices, 0.01 ticks, invalid ticks and low prices that would round to zero. Each test should explain its expected outcome.

Organize suites without hiding failures

Register markers such as `api`, `browser`, `slow` and `security` in pytest configuration. `--strict-markers` catches spelling mistakes. `skip` records a test not executed; `xfail` records a known expected failure. Use `xfail(strict=True)` for tracked defects when an unexpected pass should demand review. Do not use `xfail` to make an unexplained financial-safety failure appear acceptable.

BASH

```

python -m pytest -m api -q
python -m pytest -m "not browser" -q
python -m pytest -x --tb=short
python -m pytest --junitxml=artifacts/results.xml

```

`-x` stops after the first failure and is useful while debugging; it is not always best for nightly discovery. JUnit XML enables CI reporting and is separate from a human-readable HTML report.

Coverage asks where tests executed

With optional `pytest-cov`, use `--cov=domain --cov-branch --cov-report=term-missing`. Branch coverage asks whether alternative paths executed. Even 100% branch coverage cannot prove all event orders, numeric ranges or authorization cases. Add mutation thinking: if `<=` becomes `<`, does a test fail? If filled quantity is ignored, does a test fail? These questions assess assertions, not merely execution.

Async tests

Choose one supported async testing approach and configure it explicitly. This recipe requires optional `pytest-asyncio`:

PYTHON

```

import asyncio
import pytest

@pytest.mark.asyncio
async def test_event_released():
    event = asyncio.Event()
    asyncio.get_running_loop().call_soon(event.set)
    await asyncio.wait_for(event.wait(), timeout=2)
    assert event.is_set()

```

Do not call `asyncio.run()` from inside an already running event loop. Match Playwright's sync API to synchronous tests and async API to asynchronous tests. Mixing them can create loop errors or blocked execution. Timeouts bound a test; they should not be its only oracle.

Practice and checkpoint

Parameterize both sides and exact target/stop thresholds. Then intentionally change a comparator in your disposable copy and observe which test catches it. Restore the code immediately. Explain why a skipped browser suite must be visible in a report claiming "all tests passed."

Chapter 17. Mocks, Fakes, Spies and Broker Contract Tests

Choose the smallest useful substitute

A stub returns canned data. A fake implements simplified behavior, such as an in-memory store. A spy records calls. A mock supports expectations about interactions. These terms overlap in libraries; the important question is what behavior the substitute preserves and what it omits.

PYTHON

```
from unittest.mock import Mock

def place_if_enabled(broker, enabled):
    if enabled:
        return broker.place(symbol="TEST", qty=2)
    return None

def test_disabled_never_submits():
    broker = Mock()
    assert place_if_enabled(broker, False) is None
    broker.place.assert_not_called()
```

Use AsyncMock for awaited dependencies. Patch where a name is looked up, not automatically where it was originally defined. If `service.py` imports `fetch` into its own namespace, patch `service.fetch`. `autospec` or `spec_set` can expose missing methods and signature mistakes. An unrestricted mock accepts almost anything and can conceal exactly the SDK mismatch you intended to detect.

Acceptance is not execution

Broker adapters should normalize vendor responses into a stable internal contract without inventing fills. A response containing an order identifier proves acceptance or registration according to the broker's contract, not necessarily traded quantity. Test a sequence: accepted -> pending -> partial 4/10 -> cancelled with final filled 4. The local position must reflect 4, and any replacement must consider only the reconciled remainder.

Official broker references: [Dhan orders](#) and [Kite orders](#). Use the installed SDK version and reviewed response fixtures; do not call a live order endpoint as part of a beginner exercise. Contract fixtures should remove account IDs and secrets while retaining field names and types.

Adversarial fixtures

Include empty bodies, malformed JSON, HTTP 429, HTTP 500, unknown status, missing filled quantity, error envelopes and unexpected additional fields. Forward-compatible parsing can ignore harmless extra fields, but missing safety-critical fields should produce an unresolved/error outcome rather than guessed success. An unrecognized rejection reason should not trigger an unconditional retry.

A mocked adapter test proves your behavior for the supplied contract. It cannot prove that a vendor still returns that contract today. A separate read-only compatibility check, version review and controlled staging validation cover that gap. Never claim that a monkeypatched success response validates network authentication.

Practice and checkpoint

Write a fake broker with a list of scripted states and a submission counter. Send the same event twice and assert one submission. Then simulate a network timeout after acceptance. The correct next step is reconcile by correlation/order identity, not assume the order failed and place another full quantity.

Chapter 18. Database, Redis, Transactions and Concurrency Tests

Test durable facts, not only responses

An API can return "saved" while the wrong Redis key was written. An integration test should read through the supported store boundary and verify user, strategy and field values. A persistence test should reopen the storage connection or restart the component as appropriate. Reusing the same in-memory object cannot prove restart recovery.

For a relational design, choose keys and constraints that encode ownership. A conceptual positions table might use account ID, strategy ID and trade ID, with symbol as an attribute rather than the only key. Test uniqueness, foreign keys, nullability, transaction rollback and migration compatibility. Use parameterized SQL; test user input containing punctuation as data, never concatenate it into a query.

SQL

```
SELECT strategy_id, symbol, remaining_qty
FROM positions
WHERE account_id = :account_id AND status = :status;
```

The named parameters here are conceptual; use your database driver's supported parameter syntax. Authorization must come from the authenticated identity, not a caller-selected account ID alone.

Redis is more than a dictionary

A fake Redis implementation is useful for fast tests, but verify critical Lua scripts, expiry, streams, acknowledgment and atomic compare-and-set behavior against a disposable real Redis instance too. Configure a unique namespace per test worker. Never run FLUSHALL on a client VM. Verify that stale lease tokens cannot release a new owner's lock and that queued jobs are not lost between claiming and completion.

Deterministic concurrency

This recipe proves an order task can finish while a simulated blocking data request remains blocked. It does not measure production latency:

PYTHON

```
import asyncio
import threading

async def demonstrate_independent_io():
    release = threading.Event()
    started = asyncio.Event()
    loop = asyncio.get_running_loop()

    def data_call():
        loop.call_soon_threadsafe(started.set)
        if not release.wait(5):
            raise TimeoutError("Test gate was not released")
```

```
task = asyncio.create_task(asyncio.to_thread(data_call))
try:
    await asyncio.wait_for(started.wait(), 2)
    order_result = await asyncio.to_thread(lambda: "accepted")
    assert order_result == "accepted"
    assert not task.done()
finally:
    release.set()
    await task
```

For distributed orders, test the dangerous window: broker accepts, worker crashes before recording the result, job is redelivered. An outbox or durable intent record can improve recoverability, but the acceptance criteria must still require reconciliation before another submission. "Exactly once" is not created by adding a queue.

Practice and checkpoint

Design tests for expired locks, stale lock owners, duplicate jobs and crash recovery. Include a test where ownership is lost immediately before broker submission. Expected behavior is to stop new submission and reconcile; do not weaken the lock to make a flaky test pass. The repository's `tests/test_order_locks.py` is a useful advanced reading exercise after you understand these invariants.

Part IV. Playwright Browser Automation

Chapter 19. The Web, DOM, Browser and First End-to-End Test

What the browser actually does

The browser downloads HTML, constructs a document tree, applies CSS and executes JavaScript. JavaScript can change the document after initial loading, send HTTP requests and receive streamed messages. The DOM is the current document representation, not necessarily the original source. A screenshot captures pixels; a locator finds an element through DOM and accessibility information.

Playwright launches a browser, creates isolated contexts and opens pages. A context is an independent browser session with its own cookies and storage. A page is a tab. The pytest plugin supplies a page fixture and handles its lifecycle. Install the browser binaries separately from the Python package. See the [official installation guide](#) for platform support.

A small, complete browser test

PYTHON

```
from playwright.sync_api import expect

def test_form_contract(page):
    page.set_content('''
        <label for="name">Strategy name</label>
        <input id="name">
        <button>Save strategy</button>
    ''')
    page.get_by_label("Strategy name").fill("alpha")
    expect(page.get_by_label("Strategy name")).to_have_value("alpha")
    expect(page.get_by_role("button", name="Save strategy")).to_be_enabled()
```

This test proves the local HTML interaction, not backend persistence. `set_content` is excellent for learning a control in isolation. For end-to-end coverage, use `page.goto(url)`, click the real save button and inspect both status message and API readback. The companion `test_browser.py` demonstrates that complete path.

A locator describes how to find an element and resolves when used. Prefer role and accessible name for buttons, labels for inputs and explicit test IDs for complex components. These choices survive many harmless layout changes. A selector built from the fifth nested `div` expresses layout coincidence rather than user intent.

Locators and assertions work together

`expect(locator).to_have_text(...)` retries until the condition succeeds or times out. `assert locator.inner_text() == ...` reads once and may fail during a legitimate update. Use Python assertions for static API data and Playwright expectations for asynchronous browser state.

Multiple matching buttons cause strictness errors for a single-element action. Scope to the relevant dialog, form or row. Do not reflexively append `.first`; that may select another strategy's action. [Locator documentation](#) explains the supported locator methods.

Practice and checkpoint

Create two Save buttons in different forms and observe the strictness error. Scope one using a form test ID or accessible group. Explain what the test proves after clicking but before verifying a response: only that an action was attempted, not that the backend committed the change.

Chapter 20. CSS, XPath, Shadow DOM and Selector Tools

Understand before generating

CSS selectors describe elements through tag, ID, class, attributes and relationships. `#config` selects an ID, `.error` selects a class, and `[data-field="pn1"]` selects an attribute. A space means descendant; `>` means direct child. Prefer attributes created as stable test contracts over styling classes likely to change.

PYTHON

```
def test_css_and_xpath(page):
    page.set_content('''
        <table><tr data-trade="paper-1">
            <td>TEST</td><td data-field="pn1">2.00</td>
        </tr></table>
    ''')
    row = page.locator('[data-trade="paper-1"]')
    assert row.locator('[data-field="pn1"]').inner_text() == "2.00"
    assert page.locator("xpath=//tr[@data-trade='paper-1']/td[1]").inner_text() == "TEST"
```

XPath navigates the document using expressions and supports parent/ancestor relationships. It can be useful in legacy markup, but absolute paths such as `/html/body/div[3]/div[2]` are fragile. CSS classes can also be fragile; neither language is automatically reliable. The stability of the chosen contract matters more than selector syntax.

Shadow DOM and frames differ

A web component may contain an open shadow root. Playwright's normal locators can work through open shadow roots; XPath does not pierce them. Closed shadow roots are not a general automation target through normal locators. An iframe contains another document and needs a frame locator. Do not treat every element-not-found error as a timeout problem: first inspect which document or shadow root owns the element.

PYTHON

```
from playwright.sync_api import expect

def test_open_shadow_component(page):
    page.set_content('<div id="host"></div>')
    page.evaluate('''() => {
        const root = document.querySelector('#host').attachShadow({mode: 'open'});
        root.innerHTML = '<button>Reconnect feed</button>';
    }''')
    expect(page.get_by_role("button", name="Reconnect feed")).to_be_visible()
```

Selector helper extensions, including SelectorsHub, may help inspect and generate selectors. Review permissions and avoid installing extensions into a browser profile holding client credentials. Generated selectors are suggestions. Check uniqueness, semantic meaning and behavior after a rerender. Do not make an extension a required dependency of the CI test run.

Practice and checkpoint

Find a control using role, CSS and XPath. Change its wrapper layout and see which selectors survive. The exercise is not to declare XPath forbidden; it is to recognize which assumptions each selector encodes. Add an explicit data-testid only when a stable accessible locator does not express the intended target well.

Chapter 21. Inputs, Radios, Checkboxes and Dropdowns

Model controls according to their behavior

Use `fill()` for a text input, `check()` or `uncheck()` for checkboxes, and `select_option()` for a native select. These actions express intent better than clicking coordinates. A toggle may be a checkbox, button with switch semantics, or a custom widget; inspect its accessibility role and use the matching contract.

PYTHON

```
from playwright.sync_api import expect

def test_native_controls(page):
    page.set_content('''
        <label>Capital<input name="capital"></label>
        <label><input type="checkbox">Trailing SL</label>
        <label><input type="radio" name="side">Long</label>
        <label>Product<select><option value="MIS">Intraday</option>
        <option value="CNC">Delivery</option></select></label>
    ''')
    page.get_by_label("Capital").fill("5000")
    page.get_by_label("Trailing SL").check()
    page.get_by_label("Long", exact=True).check()
    page.get_by_label("Product").select_option("CNC")
    expect(page.get_by_label("Trailing SL")).to_be_checked()
    expect(page.get_by_label("Product")).to_have_value("CNC")
```

The [actions guide](#) documents these APIs. In your app, verify not only the control state but also the saved boolean. A checked appearance with a string-valued false can become true in weak backend parsing.

Native versus Bootstrap-style dropdowns

A custom dropdown is often a button opening a list of clickable items. It is not a `<select>`, so `select_option()` is inappropriate. Open the menu, wait for the visible option, choose it, and assert the selected value. Hidden duplicate menus can cause strict locator errors; scope to the visible menu or the component that owns it.

Recipe for a component exposing listbox/option semantics:

PYTHON

```
def choose_custom_product(page):
    page.get_by_role("button", name="Choose product").click()
    menu = page.get_by_role("listbox", name="Products")
    menu.get_by_role("option", name="Delivery", exact=True).click()
```

Do not use `force=True` to click a hidden option just to make a test pass. That bypasses an important user condition. A hidden menu or disabled setting may be behaving correctly because its parent toggle is off.

The executable `test_custom_dropdown` in `lab/tests/test_widgets.py` builds a listbox, opens it, chooses Delivery and verifies both the selected value and menu closure. Read its HTML alongside the test. Notice that the test does not assume the component is implemented with Bootstrap; it targets the same observable control contract regardless of styling library.

Dependent settings

For breakout controls, test toggle off -> timeframe and TTL unavailable; toggle on -> enabled; configure; save; reload; values retained. For trailing stop, toggle off must disable trailing behavior, not merely grey out a field. That second requirement belongs in a backend risk test. This is how frontend and backend responsibilities complement each other.

Practice and checkpoint

Test a zero buffer and a disabled buffer separately. They may currently produce similar numerical thresholds but represent different saved configurations. Your expected result should state both the persisted flag and the value, not only the final price.

Chapter 22. Static Tables, Dynamic Rows and Pagination

Row identity matters

A trading dashboard may show several alerts for the same symbol. A symbol alone is not a sufficient identity for every action. Use strategy plus trade ID or a dedicated row identifier. Scope child locators to the selected row so an assertion about one order does not read another order's P&L.

Recipe for the lab's table:

PYTHON

```
from playwright.sync_api import expect

def expect_paper_position(page, strategy, symbol, pnl):
    table = page.get_by_role("table", name="Positions")
    row = table.get_by_role("row").filter(has_text=strategy).filter(has_text=symbol)
    expect(row).to_have_count(1)
    expect(row.locator('[data-field="pnl"]')).to_have_text(pnl)
```

For ambiguous substring names such as alpha and alpha-long, use exact cells or a trade ID. The helper above is appropriate only when its fixture guarantees unique matching text. A careful helper documents that precondition rather than silently calling `.first`.

Rerendering and stale assumptions

After a tick or refresh, the application may replace the entire table body. Locators re-resolve; cached element handles can point to detached nodes. Prefer locators for ongoing interaction. If ordering changes with P&L, do not assert that the first row remains a particular symbol unless sorting itself is the feature under test.

The historical bug in this project is instructive: a skipped alert row became LIVE when a later position existed for the same symbol. A regression test should seed an earlier skipped alert and a later successful trade, then assert their statuses independently. It should also verify that P&L updates do not attach to the skipped row and that only the live row has a square-off control.

Pagination and virtualized lists

For numbered pagination, verify current-page indicator, row range, next/previous transitions and final-page disabled state. For cursor pagination, assert no duplicated or missing IDs across the collected pages. Sorting or filtering should reset the page or follow an explicit policy. An empty filtered result must not display stale rows.

A virtualized table renders only visible rows. Counting DOM rows does not count the full dataset. Test data-source totals through an API contract and test visible rendering separately. Scroll through representative segments and assert stable identity. Avoid scraping thousands of rows when an API test can validate the underlying dataset more cheaply.

Run the companion `test_pagination_identity`. It uses three synthetic pages and a bounded loop. Every iteration checks the expected row, records its identity and moves only when another page is expected. The final assertions verify the last-page button is disabled and all expected identities were visited exactly once. Extend it with a filter that resets pagination and a deliberately duplicated record to see the failure.

Practice and checkpoint

Create a fixture with two strategies owning the same symbol and one rejected historical alert. Write three row assertions. Explain why selecting the first matching symbol is insufficient for choosing the exit button safely.

Chapter 23. Date Pickers, Dialogs and Frames

Dates have both representation and meaning

A native date input accepts an ISO-style value such as `2026-09-23`. The displayed text depends on browser locale. Assert the value rather than assuming the rendered order is day/month/year. A custom jQuery or Bootstrap calendar may require opening a panel, navigating month/year and selecting a day button. Scope the day to the intended month because adjacent months can show duplicate day numbers.

PYTHON

```
from playwright.sync_api import expect

def test_native_date(page):
    page.set_content('<label>From date<input type="date"></label>')
    page.get_by_label("From date").fill("2026-09-23")
    expect(page.get_by_label("From date")).to_have_value("2026-09-23")
```

For a custom calendar, write a bounded navigation loop or select month/year through supported controls. Do not loop forever until text happens to match. Test invalid ranges, leap days, start after end, timezone conversion and a range including a market holiday. The calendar widget cannot establish whether a broker has candle data for the selected session.

The companion `test_custom_calendar_scopes_duplicate_day` intentionally contains two buttons labelled 23 in different month regions. A global locator would be ambiguous. Scoping to September selects the intended date and verifies the stored ISO value. This small example isolates the central problem in larger jQuery/Bootstrap calendars without requiring an external demo website.

Browser dialogs versus HTML modals

alert, confirm and prompt are browser dialogs. An HTML modal is ordinary page content and should be located by its dialog role or container. Register a browser dialog handler before the action that opens it:

PYTHON

```
def test_confirm_accept(page):
    page.set_content('''<button onclick="document.body.dataset.ok=confirm('Close paper?')">
        Close paper position</button>''')
    page.once("dialog", lambda dialog: dialog.accept())
    page.get_by_role("button", name="Close paper position").click()
    assert page.locator("body").get_attribute("data-ok") == "true"
```

Write a separate dismiss test and verify no mutation occurs. A handler that observes but never accepts or dismisses can stall the action. The [dialog guide](#) describes this lifecycle.

Frames contain another document

PYTHON

```
from playwright.sync_api import expect

def test_frame(page):
    page.set_content('''<iframe title="Paper help"
        srcdoc="<button>Read risk notice</button>"></iframe>''')
    button = page.frame_locator('iframe[title="Paper help"]').get_by_role("button")
    expect(button).to_have_text("Read risk notice")
```

Use [frame locators](#) rather than assuming a top-level locator searches another document. Real broker authentication can involve redirects or external frames; mock the boundary in routine CI and never automate a real customer's PIN as a learning exercise.

Practice and checkpoint

Test accepting and cancelling a paper-only close dialog. Explain why a browser confirmation dialog cannot be found with `page.locator('.modal')`. They are different UI mechanisms with different automation APIs.

Chapter 24. Mouse, Keyboard, Uploads and Downloads

Interactions should express user intent

Use `hover()` for tooltips, `dblclick()` for a deliberate double-click action, and `click(button='right')` for a context menu. A drag operation needs a defined source and target; use `drag_to()` where appropriate and assert the resulting order or value. Coordinate clicks are a last resort for surfaces such as canvases and should be tied to stable geometry.

Keyboard tests reveal accessibility and focus defects. Tab, Enter and Escape test navigation and dismissal. Use `ControlOrMeta` for platform-aware select-all shortcuts when supported by the installed version. `fill()` is preferable for normal text entry; sequential typing is useful only when the feature depends on individual key events.

Upload files without operating the OS dialog

PYTHON

```
def test_upload(page, tmp_path):
    file = tmp_path / "symbols.csv"
    file.write_text("symbol,qty\nTEST,2\n", encoding="utf-8")
    page.set_content('<label>Import symbols<input type="file"></label>')
    control = page.get_by_label("Import symbols")
    control.set_input_files(file)
    assert control.evaluate("el => el.files.length") == 1
    assert control.evaluate("el => el.files[0].name") == "symbols.csv"
```

Upload tests should cover file type, size, missing headers, malformed records and partial import policy.

Client-side validation alone is insufficient; API tests must bypass it and verify backend rejection. Never test malware against an unauthorized public system. Use harmless synthetic strings to test sanitization contracts.

Downloads need both event and content checks

Recipe for a page with an Export CSV control:

PYTHON

```
import csv

def check_export(page, tmp_path):
    with page.expect_download() as pending:
        page.get_by_role("button", name="Export CSV").click()
    download = pending.value
    target = tmp_path / "positions.csv"
    download.save_as(target)
    with target.open(newline="", encoding="utf-8") as stream:
        rows = list(csv.DictReader(stream))
    assert rows[0]["symbol"] == "TEST"
```

Register the wait before clicking so a fast download is not missed. Choose your own safe destination rather than trusting a remote filename as a filesystem path. Verify headers, records, encoding and formula-injection handling when spreadsheets may open exported values. The [download guide](#) covers the event API.

Practice and checkpoint

Test an empty CSV and a valid two-row CSV. Decide whether one invalid row rejects the whole import or reports partial success; both need explicit acceptance criteria. Verify that a failed upload does not silently create half a strategy configuration.

Chapter 25. Browser Contexts, Tabs and Authentication State

Isolate sessions deliberately

A new page in the same context shares cookies. A new context has a separate session. Use two contexts to test that one account cannot see another account's data; two tabs in one context do not provide that isolation. Context storage state is convenient for authenticated tests but is also a credential-bearing artifact.

Recipe for a paper UI that opens help in a new tab:

PYTHON

```
from playwright.sync_api import expect

def open_help(page):
    with page.expect_popup() as pending:
        page.get_by_role("link", name="Help").click()
    help_page = pending.value
    expect(help_page.get_by_role("heading", name="Risk guide")).to_be_visible()
```

Register popup waits before triggering them. Use the returned page, not an assumption about `context.pages[1]`, because unrelated tabs may exist. Closing a popup should not log the original page out unless that is an explicit application policy.

Reuse login carefully

After a synthetic test login, save state to an ignored temporary path. Start another context with that state and verify access. Also test expiry and logout: stale cookies must no longer grant protected access. [Playwright authentication guidance](#) explains storage-state reuse and its sensitivity.

For admin/TOTP in the real project, routine UI tests should use a test identity and controllable authenticator time in an isolated environment. Do not disable production authentication to make a curl command easier. A terminal request does not inherit the session cookie from your browser, explaining why an already logged-in user can still receive `ADMIN_AUTH_REQUIRED` from curl.

Multi-user and cross-strategy boundaries

Test account A requesting account B's positions with a modified query parameter. Expect denial or enforced A-only scope according to the API design. Separately test strategy A's exit alert against strategy B's symbol. Authentication answers who you are; authorization answers what you may do. A single-admin app still needs authorization on sensitive API routes because someone can call them without using the dashboard.

Practice and checkpoint

Open two independent contexts with no real credentials and use a local cookie demonstration. Set a cookie in one, assert it is absent in the other. Then use two pages in the first context and observe sharing. Explain why worker-specific accounts or per-test state are necessary for parallel authenticated tests that modify settings.

Chapter 26. Waiting, Screenshots, Video, Traces and Flaky Tests

Wait for evidence, not arbitrary time

Page navigation can finish before a dashboard is useful. `networkidle` is especially unsuitable as a universal readiness signal for polling or streaming apps. Wait for a meaningful state: a configuration heading, a loaded form, or a particular response. A button being visible does not prove its data was loaded.

PYTHON

```
from playwright.sync_api import expect

def save_and_check(page):
    with page.expect_response(lambda r:
        r.url.endswith("/api/strategies") and r.request.method == "POST"
    ) as pending:
        page.get_by_role("button", name="Save strategy").click()
    assert pending.value.status == 201
    expect(page.get_by_role("status")).to_have_text("Strategy saved")
```

This recipe uses the lab endpoint. With production routes, update both the URL and documented success status. Waiting for any response with a similar path can accidentally match a GET rather than the save POST.

Preserve failure evidence**BASH**

```
python -m pytest -m browser --tracing retain-on-failure --screenshot only-on-failure --video retain-on-failure
python -m playwright show-trace test-results/PATH/trace.zip
```

Use the actual generated trace path. These pytest plugin options apply to plugin-managed contexts; a context you create manually may require explicit recording and closure. Screenshots capture final appearance, video shows visible timing, and traces show actions, DOM snapshots and network information. See the [trace viewer](#). Restrict artifact access and retention because requests and screenshots may contain private data.

Classify flakiness

Common sources are shared state, unstable selectors, missing waits, genuine races, variable resources and external dependencies. Record repeat frequency, environment and the first failing assertion. Re-running until green hides evidence. A bounded diagnostic rerun can distinguish reproducibility, but release reporting must keep the original failure visible.

In this project's browser smoke test, checking a JavaScript function before dashboard initialization produced intermittent undefined-function errors. The fix was explicit readiness, not changing financial behavior. Another test demanded a 50 ms scheduling result on a busy workstation. Replacing that assertion with event-controlled ordering tested concurrency more reliably; a separate benchmark is needed for latency claims.

Practice and checkpoint

Create a button that displays a message after a short asynchronous delay. Write one brittle immediate assertion, then replace it with an expectation. Explain why increasing every timeout to two minutes would slow diagnosis without fixing a wrong selector or broken server response.

Chapter 27. Network Mocking and WebSocket Evidence

Mock a boundary, not the whole feature

Routing lets a test supply a controlled response or abort a request. Use it to test frontend behavior for server errors, delays and malformed content. It does not test the real backend implementation for that endpoint. Keep complementary API tests so a mocked browser suite cannot pass while the real route is broken.

PYTHON

```
from playwright.sync_api import expect

def test_read_failure_visible(page, lab_url):
    page.route("**/api/positions", lambda route: route.fulfill(
        status=502, content_type="application/json", body='{"detail": "UPSTREAM_DOWN"}'
    ))
    page.goto(lab_url)
    expect(page.get_by_role("status")).to_have_text("UPSTREAM_DOWN")
```

Also test an HTML error page. The frontend should present a useful safe message instead of a raw JSON parse error. Our deliberately small lab can be extended for this assignment; the production dashboard's fallback behavior must be tested through its own route. [Network documentation](#) describes interception and event observation.

Two separate WebSocket tests

A broker-stream test feeds normalized or recorded synthetic packets into the market-feed adapter and verifies security-ID mapping, source, timestamp and downstream publication. A dashboard-stream test sends an app message to a connected browser and checks LTP/P&L rendering. Neither alone proves the full broker-to-risk-to-browser chain. Add a system test that joins those boundaries using a fake broker transport and disposable shared store.

Recipe requiring a page that actually creates a WebSocket:

PYTHON

```
def capture_socket_frames(page, lab_socket_url):
    frames = []
    def on_socket(socket):
        socket.on("framereceived", lambda payload: frames.append(payload))
    page.on("websocket", on_socket)
    page.goto(lab_socket_url)
    return frames
```

The compact companion lab uses manual refresh and does not implement this endpoint. A robust stream test waits for a particular validated frame and UI outcome rather than immediately inspecting an initially empty list.

Test failure sequences

Cover connect without ticks, subscription before credentials, symbol added after connection, reconnect preserving subscriptions, stale timestamp, out-of-order tick and malformed packet. A restart should not create an unlimited number of threads or event loops. A REST fallback price must remain labelled as fallback; it must not falsely certify the WebSocket healthy.

Practice and checkpoint

Build a timeline: subscription requested at $t=0$, transport connected at $t=1$, tick received at $t=3$, UI updated at $t=3.1$. Name the evidence required at each step. Then delay the tick past the freshness budget. Expected behavior must follow the application's configured fallback/block policy, not an invented guarantee of instant execution.

Chapter 28. Mobile, Cross-Browser and Parallel Testing

A viewport is only one dimension

Test desktop, tablet and mobile widths with the same important workflows: configure, save, reload, observe a position and inspect an error. Check that text fits, controls remain reachable, tables scroll predictably and the document has no unintended horizontal overflow. Use DOM geometry assertions together with screenshots. A screenshot can look fine while a covered button is unclickable.

PYTHON

```
import pytest
from playwright.sync_api import expect

@pytest.mark.parametrize("size", [(1280, 900), (768, 1024), (390, 844)])
def test_responsive_form(page, lab_url, size):
    page.set_viewport_size({"width": size[0], "height": size[1]})
    page.goto(lab_url)
    expect(page.get_by_role("button", name="Save strategy")).to_be_visible()
    assert page.evaluate("document.documentElement.scrollHeight <= innerWidth")
```

Viewport changes are not full device emulation. Touch, device scale factor, user agent, virtual keyboard and actual hardware can reveal additional issues. Browser automation cannot claim that a site works on every phone. Include manual checks on representative real devices where the audience requires them.

Browser engines and parallel workers

BASH

```
python -m playwright install chromium firefox webkit
python -m pytest -m browser --browser chromium --browser firefox --browser webkit
python -m pytest -m browser -n 2 --browser chromium
```

The second command runs the selected engines using the pytest plugin. The third requires `pytest-xdist`. More workers may make a small machine slower due to memory and CPU contention. Measure total duration and failures before increasing parallelism. The [plugin reference](#) documents browser and worker options.

Each worker needs independent server state, data identities and artifact paths. The companion `lab_url` binds an available port and creates a fresh app per test. A shared fixed port or a global demo record on one server would cause cross-worker interference. Do not share a mutable browser Page across threads.

Practice and checkpoint

Run the lab browser suite serially and with two workers. If parallel execution fails, inspect ports, shared IDs and fixture scope before adding retries. Record which engines actually ran; a Chromium-only result is not cross-browser evidence, even if WebKit was installed successfully.

Part V. Framework Design and API Automation

Chapter 29. Page Object Model and Framework Design, Part One

Build after you understand repeated behavior

A framework is an organized set of conventions, helpers and fixtures that make tests easier to write and maintain. It should not require a beginner to understand twenty abstractions before writing one assertion. Start with explicit tests, identify real duplication, then extract small helpers. Page Object Model groups page interactions behind meaningful methods.

PYTHON

```
from playwright.sync_api import expect

class StrategyPage:
    def __init__(self, page):
        self.page = page
        self.name = page.get_by_label("Strategy name")
        self.qty = page.get_by_label("Quantity")

    def save(self, name, qty):
        self.name.fill(name)
        self.qty.fill(str(qty))
        self.page.get_by_role("button", name="Save strategy").click()
        expect(self.page.get_by_role("status")).to_have_text("Strategy saved")

def test_save_with_page_object(page, lab_url):
    page.goto(lab_url)
    strategy = StrategyPage(page)
    strategy.save("alpha", 2)
    saved = page.request.get(lab_url + "/api/strategies").json()
    assert saved[0]["qty"] == 2
```

This helper contains locators and one meaningful operation. It does not decide every business assertion for the test. Some teams keep all assertions in tests; others allow page objects to assert readiness and successful completion of their own operations. Choose a consistent rule. The important point is not to hide the expected business outcome behind an always-passing helper.

A small architecture

TEXT

tests/	scenarios and expected outcomes
pages/	browser interactions
clients/	HTTP transport helpers
data/	synthetic input sets and schemas
conftest.py	lifecycle and dependency wiring
pytest.ini	markers and reporting defaults
artifacts/	ignored evidence, not source

Keep domain expectations independent from the implementation under test. Importing the production target calculator to generate every expected target means the test may duplicate the bug rather than detect it. Use hand-worked examples or an independently specified property for critical values.

Common overengineering traps

A universal `click_element(locator_type, locator, retries, timeout)` wrapper can obscure Playwright's built-in behavior. A deeply inherited `BasePage` can force unrelated pages to depend on login state. A retry decorator around every operation can duplicate side effects. Prefer direct Playwright calls until a real cross-cutting need justifies a wrapper.

Keep helpers honest about failure. Do not catch every exception and return false if callers forget to assert that result. Raise with useful context. Avoid helpers that call production APIs based on an unvalidated environment variable. A framework should fail closed when its target is not the intended lab/staging environment.

Practice and checkpoint

Extract one repeated configuration operation into a page object, then change a label in the lab HTML and update one locator. Explain which assertions remain in the test and why. A good explanation distinguishes page readiness from business persistence.

Chapter 30. Framework Design, Parts Two and Three: Data, Reports and Maintenance

Configuration must not become a secret leak

Separate nonsecret settings, synthetic datasets and credentials. Nonsecret settings include base URL, browser and timeouts. Test identities and tokens belong in an approved secret mechanism or temporary fixture. Environment variables are a delivery mechanism, not encryption. Never print an entire environment to a CI log.

For a learning suite, validate that the base URL is loopback before any request. Use an explicit environment selection for staging. Do not accept an arbitrary public URL and assume it is safe because the test is called smoke.

PYTHON

```
from urllib.parse import urlsplit

def require_local_url(url):
    parsed = urlsplit(url)
    if parsed.scheme != "http" or parsed.hostname not in {"127.0.0.1", "localhost"}:
        raise ValueError("Learning tests require a local HTTP server")
    if parsed.username or parsed.password:
        raise ValueError("Credentials must not be embedded in test URLs")
    return url
```

This is a teaching safeguard, not a complete SSRF defense for a public service. Production authorization and network policies are separate concerns.

Reports should answer release questions

JUnit XML serves CI aggregation. Optional `pytest-html` generates an HTML report. Optional `allure-pytest` records richer test steps and attachments; the Allure command-line tool must also be installed to render its results. Consult the [Allure pytest guide](#) for compatible setup rather than assuming the Python package alone includes the CLI.

BASH

```
python -m pytest --junitxml=artifacts/junit.xml
python -m pytest --html=artifacts/report.html --self-contained-html
python -m pytest --alluredir=artifacts/allure-results
allure generate artifacts/allure-results -o artifacts/allure-report --clean
```

Run the report commands only after installing their optional plugins. Generate each run into a new directory or clean only the dedicated previous report directory. Old result files can inflate counts or confuse failures. Reports should identify commit, environment, start/end time, tests collected, passed, failed, skipped and any expected failures.

Framework maintenance

Review test code like product code. Require readable names, deterministic setup, meaningful assertions and cleanup. Track flaky tests with owners and deadlines. Quarantine is temporary containment, not deletion of evidence. Keep the fastest useful checks on every pull request and schedule slower browser matrices separately. Prefer a small reliable release gate to thousands of meaningless clicks.

Practice and checkpoint

Generate JUnit XML from the lab and locate one test name, duration and outcome. Intentionally fail a test in a disposable branch and confirm CI would fail even if report generation succeeds. A report is evidence of a run, not authority to turn a red run green.

Chapter 31. HTTP, REST and CRUD API Testing

Understand a request before automating it

An HTTP request has a method, URL, headers and optional body. A response has status, headers and optional body. GET typically reads; POST commonly creates or triggers an operation; PUT commonly replaces a resource; PATCH modifies selected fields; DELETE removes it. These are design conventions, and your API contract determines precise semantics. Do not invent unsupported methods because a course listed them.

The lab uses POST to save a strategy, GET to list strategies, DELETE to remove one and POST to submit synthetic signals/ticks. It does not implement PUT or PATCH. Add them as a deliberate exercise with documented idempotency and partial-update semantics.

In-process API tests**PYTHON**

```
def test_strategy_crud(client):
    created = client.post("/api/strategies", json={"name": "alpha", "qty": 2})
    assert created.status_code == 201
    assert created.json()["qty"] == 2
    listed = client.get("/api/strategies")
    assert listed.status_code == 200
    assert listed.json()[0]["name"] == "alpha"
    deleted = client.delete("/api/strategies/alpha")
    assert deleted.status_code == 204
    assert client.get("/api/strategies").json() == []
```

FastAPI's TestClient exercises routing, validation and application code without opening a real network socket. It does not test Nginx, TLS, DNS or a deployed process. Use a second layer of network tests for those boundaries. Never parse JSON from a 204 response with no body.

Playwright APIRequestContext

PYTHON

```
def test_network_api(playwright, lab_url):
    api = playwright.request.new_context(base_url=lab_url)
    try:
        response = api.post("/api/strategies", data={"name": "api-demo", "qty": 2})
        assert response.status == 201
        assert response.json()["name"] == "api-demo"
    finally:
        api.dispose()
```

This uses real localhost HTTP. Playwright serializes dictionary data as JSON for this request. The [API testing guide](#) explains request contexts. An independent request context has its own cookie storage; a page/context-associated request context shares the corresponding browser context's cookies.

Negative cases and semantics

Test missing fields, unknown fields, wrong types, nulls, negative values and excessive sizes. Assert a non-success status, a stable error contract and no write. A 200 response containing an error object is not automatically success; validate both transport and domain result where an external API uses that pattern. Retrying POST after a timeout requires idempotency or reconciliation because the original request may have succeeded.

Practice and checkpoint

Send a signal before saving its configuration. Expect CFG_MISSING and no position. Then save configuration and repeat with a new event ID. Explain why checking only status 200 on a create request would miss a wrong saved quantity.

Chapter 32. Headers, Cookies and JSON Schema Validation

Headers influence interpretation

Content-Type declares a body representation. Accept requests a response representation. Authorization headers carry credentials. Cache-control affects whether sensitive or stale responses can be reused. Request/correlation IDs help follow a transaction through services. HTTP header names are case-insensitive, but application tokens and many values are not.

Cookies may carry a session identifier. Test intended Secure, HttpOnly and SameSite attributes in the HTTPS deployment configuration. HttpOnly prevents ordinary script access to the cookie; it does not eliminate CSRF or all XSS risks. SameSite reduces some cross-site request contexts but is not a complete authorization policy. Do not paste live cookies into test source.

Schema and business meaning are different

JSON Schema checks structure, required fields, types and constraints. It cannot alone establish that the P&L is mathematically correct or that the position belongs to the authenticated user. Pair schema validation with semantic assertions.

PYTHON

```
from jsonschema import validate

POSITION_SCHEMA = {
    "type": "object",
    "required": ["strategy", "symbol", "qty", "status", "pnl"],
    "properties": {
        "strategy": {"type": "string", "minLength": 1},
        "symbol": {"type": "string"},
        "qty": {"type": "integer", "minimum": 1},
        "status": {"enum": ["OPEN", "CLOSED"]},
        "pnl": {"type": "string"},
    },
    "additionalProperties": True,
}

def assert_position_schema(position):
    validate(position, POSITION_SCHEMA)
```

This schema is for the compact lab, where closed positions preserve their original quantity for reporting. Do not reuse it unchanged for a production schema whose remaining quantity becomes zero or whose states include PARTIAL/EXITING. Schema strictness should distinguish safety-critical omissions from harmless additive vendor fields.

Contract evolution

Store sanitized fixtures with a source/version note. When a vendor changes a field from numeric to string, decide whether the adapter should normalize it or reject it. Test unknown statuses explicitly. Upgrading a Python SDK can change method signatures even when REST field names remain unchanged; unit mocks without a spec may miss this. Record installed package versions in compatibility reports.

Practice and checkpoint

Remove status from a lab position response and observe schema failure. Then set pnl to a wrong but valid string. Schema should pass, showing why a separate arithmetic assertion is necessary. Write the second assertion using independently calculated expected P&L.

Chapter 33. Authentication, Email Verification, TOTP and Authorization

Different credentials solve different problems

Basic authentication sends a username/password representation in a header and requires transport protection; encoding is not encryption. Bearer-token authentication grants access to whoever holds a valid token. API keys identify an integration according to the service's policy. OAuth-style delegated flows involve redirects and token issuance, with details specific to the provider. Do not assume every provider implements every OAuth feature.

Recipe for a **local authenticated training endpoint you add**, not the current unauthenticated lab:

PYTHON

```
def authorized_context(playwright, base_url, synthetic_token):
    return playwright.request.new_context(
        base_url=base_url,
        extra_http_headers={"Authorization": f"Bearer {synthetic_token}"},
    )
```

The caller must dispose of this context. Test no token, malformed token, expired token, wrong audience/account and valid token. Never derive authorization solely from `user_id` in the query string. A login form hidden in the UI is not protection for direct API calls.

Admin setup and email verification

A single-admin application has a first-run ownership problem: who is allowed to become the administrator? Require a controlled bootstrap mechanism, such as an allowlisted email plus a backend-issued one-time setup capability. Merely being first to open a public URL is unsafe. Test concurrent setup requests: only one may claim ownership atomically. Changing an environment email should not silently transfer an already configured account without a documented reset procedure.

For email verification, use a local mail sink or fake mail adapter, never a real customer's mailbox. Verify a random, expiring, single-use token; bind it to the intended purpose and identity. Store an appropriate protected token representation. Test reuse, expiry, wrong user, changed email and throttling. Responses should avoid unnecessary account enumeration. Email ownership verification is not automatically a second authentication factor and does not prove the person's legal identity.

TOTP lifecycle

TOTP uses a shared secret and time-based codes. A test should cover enrollment, confirming the first code, required second factor on later logins, incorrect code, expired code, allowed clock-skew window, replay policy, rate limiting and administrative reset. Use a synthetic secret and an injected clock. Protect the stored secret because validation needs it; password hashes and encrypted TOTP secrets serve different purposes.

For password handling, rely on maintained password-hashing libraries and current security guidance rather than inventing encryption. Test that responses and logs do not expose plaintext passwords, hashes, recovery codes or QR payloads. A password reset or TOTP reset should revoke relevant sessions according to a clear policy.

Redirect-based broker login tests

Mock consent generation and token consumption. Test success, denial, missing token, replay, mismatched account and session binding. Validate redirects against an allowlist and protect against login CSRF using the mechanisms supported by the provider and application session. Do not fabricate state parameters a provider does not return; inspect the real documented flow and test the application's actual binding strategy.

Practice and checkpoint

Create an authentication test matrix with five positive and ten negative cases. Include direct API access before login and after logout. The compact lab intentionally lacks authentication; this chapter is an extension assignment and production test-design guide, not a claim that its public-looking test UI is secure to deploy.

Chapter 34. Frontend-to-Backend Integration and Security Testing

Follow one fact across the system

A powerful integration test creates a configuration through the browser, verifies it through the API, submits a synthetic signal through the API, then verifies the resulting position through the browser. This crosses meaningful boundaries while avoiding repetitive UI setup for every state. The companion lifecycle test follows this pattern and checks a target exit after a synthetic tick.

Write the chain before coding:

TEXT

```
UI form qty=2 -> POST configuration -> backend stores qty=2
API signal -> synthetic position qty=2 -> GET positions
UI refresh -> OPEN row -> API tick at target
UI refresh -> CLOSED/TARGET row with P&L 2.00
```

Then introduce one failure at a time: invalid configuration, missing configuration, duplicate event, unknown symbol, price unavailable, delayed candle, order pending or rejected. Assert both the visible explanation and the absence of incorrect state. A rejected order should not acquire a full filled quantity because the UI wanted a green row.

Security is a collection of boundaries

Use the [OWASP Web Security Testing Guide](#) as a structured reference. Test only systems you own or are authorized to assess. Begin with access control, input validation, session lifecycle, output encoding, secrets exposure, CSRF defenses and rate limits. Destructive or high-load tests belong in a disposable environment with explicit scope.

For HTML output, seed a harmless string containing markup and assert it renders as text rather than executable DOM. For database queries, test punctuation through parameterized data paths. For webhook authentication, test absent and wrong secrets, replay policy and payload limits. Mask secrets in URLs because reverse-proxy logs can capture query strings.

Deployment boundaries

An in-process test cannot detect missing Nginx WebSocket upgrade headers. A deployment test should check the external HTTPS endpoint, authenticated browser connection and actual message delivery. A listening port is not proof of a working handler; an empty upstream reply can still cause a 502. Inspect application logs, reverse-proxy logs and health state together, with timestamps converted consistently.

Do not automatically reset production Redis or restart all services when a test fails. Preserve evidence, understand the failure domain, and protect open positions according to the operational runbook. Test code must not be granted broad production authority merely because it is convenient.

Practice and checkpoint

Write a test for a failed configuration save followed by page reload. Expected: no success message and no falsely persisted new values. Then test that an earlier skipped alert remains skipped after a later position for the same symbol appears. These tests connect frontend correctness to backend facts rather than treating screenshots as sufficient evidence.

Part VI. Delivery, Reliability and SDET Practice

Chapter 35. Git, GitHub and Reviewable Test Changes

Version the reasoning as well as the code

Git records project history; GitHub hosts repositories and collaboration workflows. A commit should represent a coherent change with its tests. For a defect fix, include the smallest reproducer, implementation and explanation of risk. Do not commit credentials, browser storage state, downloaded customer reports, virtual environments or giant failure videos.

Recipe in your own practice repository, not a command to publish this client's code:

BASH

```
git status
git switch -c test/breakout-expiry
git diff
git add tests/test_breakout.py
git diff --cached
git commit -m "Test breakout expiry at the exact deadline"
```

Inspect the staged diff before committing. `git add .` can include unrelated files or secrets. A `.gitignore` helps with generated artifacts but does not remove a secret already committed. If exposure occurs, revoke/rotate the secret and follow a history-cleanup policy; deleting it in the next commit alone does not revoke access.

Pull request review

A good test PR explains requirement, reproduction, changed behavior, verification and limitations. Reviewers should ask whether the test would fail before the fix, whether it uses an independent oracle, whether data is isolated, and whether cleanup runs on failure. Check that a mock did not remove the exact boundary implicated in the defect.

Merge conflicts in tests are not solved by always accepting one side. If two branches add different cases to the same parametrized list, both may be necessary. Read surrounding behavior and rerun the affected suite. Avoid rebasing or force-pushing shared history without team agreement.

Learning from repository history

Use `git log --oneline`, `git show COMMIT` and `git blame` to understand why a rule exists. A blame result identifies the most recent line change, not moral responsibility. A formatter may own the line while an older commit introduced the behavior. Read related tests and issue context.

Practice and checkpoint

Create a branch, add a regression test for a zero retry value, and write a PR description with five fields: symptom, cause hypothesis, test evidence, behavior change and remaining risks. Do not claim a live broker fix based solely on a fake response.

Chapter 36. GitHub Actions and Jenkins Pipelines

CI repeats a documented local process

Continuous integration runs checks for proposed changes in a controlled environment. It should start with dependency installation, collection/syntax checks, unit and API tests, then selected browser tests. Keep deployment permissions separate from test execution. Untrusted pull requests must not receive production secrets or privileged self-hosted access.

This GitHub Actions recipe assumes the standalone companion lab is the repository root. For the full trading repository, set a working directory and artifact paths deliberately. Review and pin action revisions according to your organization's supply-chain policy before production use.

YAML

```
name: paper-lab-tests
on: [push, pull_request]
permissions:
  contents: read
jobs:
  test:
    runs-on: ubuntu-latest
    timeout-minutes: 15
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.12'
      - run: python -m pip install -r requirements.txt
      - run: python -m playwright install --with-deps chromium
      - run: python -m pytest --browser chromium --junitxml=artifacts/junit.xml --tracing retain-on-failure
      - uses: actions/upload-artifact@v4
        if: always()
        with:
          name: test-evidence
          path: |
            artifacts/
            test-results/
          retention-days: 7
```

The version tags above are example compatible action major versions, not a claim they are the newest. A mature pipeline pins reviewed immutable revisions and updates them deliberately. The [GitHub Python CI guide](#) explains the workflow structure. Store a reviewed dependency lock for repeatable builds; the lab ranges are a starting point.

Jenkins declarative pipeline

This recipe requires a Linux agent with Python, browser OS dependencies and the standalone lab checked out. Installing OS packages should be an agent-image responsibility where possible, not unrestricted sudo granted to tests.

GROOVY

```
pipeline {
  agent { label 'python-browser' }
  options { timeout(time: 15, unit: 'MINUTES') }
  stages {
    stage('Checkout') { steps { checkout scm } }
    stage('Environment') {
      steps {
        sh 'python3 -m venv .venv'
        sh '.venv/bin/python -m pip install -r requirements.txt'
        sh '.venv/bin/python -m playwright install chromium'
      }
    }
    stage('Test') {
      steps {
        sh '.venv/bin/python -m pytest --junitxml=artifacts/junit.xml'
      }
    }
  }
  post {
    always {
      junit allowEmptyResults: false, testResults: 'artifacts/junit.xml'
      archiveArtifacts allowEmptyArchive: true,
        artifacts: 'artifacts/**,test-results/**'
    }
  }
}
```

Consult [Jenkins pipeline syntax](#) for agent and plugin requirements. A shell failure must fail the stage. `allowEmptyArchive` for optional screenshots must not become an excuse for accepting no test results. Publish evidence even on failure, with sensitive data redacted and retention limited.

Practice and checkpoint

Run a deliberately failing assertion in a disposable branch. Confirm CI is red and the report is still available. Then simulate a missing dependency and distinguish installation failure from a failed application test. Record which stage owns each failure.

Chapter 37. Codegen, Playwright MCP, PyCharm and Copilot

Generation accelerates discovery, not judgment

Playwright Codegen records browser interactions and suggests locators. Start against the local paper lab only:

BASH

```
python -m playwright codegen --target python-pytest http://127.0.0.1:8019
```

Inspect the generated code. Add assertions for backend persistence and important negative outcomes. Replace accidental selectors and remove unnecessary clicks. The recorder knows what you did, not why the result is correct. A recorded test that logs in with a real password may embed that password in generated code. Use synthetic identities and review before saving or sharing. See [Codegen documentation](#).

What MCP adds

Model Context Protocol lets an AI client use tools exposed by a server. The [official Playwright MCP server](#) enables browser interaction through that protocol. It is distinct from the Python pytest runner. An AI-guided exploration can discover a workflow; deterministic pytest tests remain the repeatable regression artifact.

A configuration recipe uses a reviewed, explicitly selected package version:

JSON

```
{
  "mcpServers": {
    "paper-browser": {
      "command": "npx",
      "args": ["@playwright/mcp@REVIEWED_VERSION"]
    }
  }
}
```

Replace REVIEWED_VERSION with a version you have reviewed before using this recipe. It is intentionally not a runnable unpinned installation command. Node.js/npm must be available for this server. Exact configuration keys and locations depend on the client; follow that client's current instructions rather than assuming one JSON file works everywhere.

PyCharm and Copilot workflow

In PyCharm, select the lab's Python interpreter and verify pytest runs normally first. Configure the desired MCP client using its settings, add the reviewed Playwright server and inspect exposed tools. JetBrains AI Assistant can act as an MCP client; PyCharm's built-in IDE MCP server is a different direction of integration. Read [JetBrains MCP client guidance](#) and [GitHub Copilot MCP guidance](#) for the installed edition/plugin.

Start a fresh browser profile with no real account cookies. Allow only the local lab workflow. Ask the assistant to inspect labels, propose test cases and explain assertions. Do not approve arbitrary shell commands or production navigation simply because a generated plan sounds confident. Page content is untrusted input and must not override your test scope.

Practice and checkpoint

Record a save-strategy flow, then add API readback and a wrong-quantity test yourself. Explain why an AI-generated green result is not sufficient without reviewing setup, assertions, side effects and the target URL. Your SDET skill is the ability to validate the automation, not merely invoke it.

Chapter 38. Performance, Resilience and Observability

Separate speed claims from correctness tests

Latency is elapsed time for an operation. Throughput is work completed per unit time. Concurrency is work in progress. Averages can hide bad tail behavior; report percentiles, error rates and queue age. Define measurement points: webhook acknowledgment, decision completion, broker acceptance, confirmed fill and UI update are different durations.

A reproducible load test specifies hardware, dependency mode, data, arrival pattern, warm-up, duration and stop conditions. Start with a fake broker and staging infrastructure. Never load-test real order APIs. A closed-loop client waits for each response before sending another; it can hide overload because it slows down when the server slows. An arrival-rate test is useful for webhook bursts, with bounded safety limits.

A measurement exercise

PYTHON

```
import time
from statistics import median

def measure(call, samples=100):
    durations = []
    for _ in range(samples):
        start = time.perf_counter()
        call()
        durations.append(time.perf_counter() - start)
    ordered = sorted(durations)
    index = max(0, int(.95 * len(ordered) + .999999) - 1)
    return {"median": median(ordered), "p95": ordered[index]}
```

This serial micro-measurement teaches percentiles; it is not a representative production load test. Record environment and payload sizes. Use a mature load tool for sustained concurrent HTTP workloads and validate its metrics independently. Do not choose a performance threshold solely because your laptop happened to achieve it once.

Recovery is part of correctness

Test Redis temporarily unavailable, broker timeout, SDK malformed response, worker termination, token rotation, stale feeds, disk pressure and graceful shutdown. Inject one fault at a time initially. Verify that no extra order is placed, unresolved state is visible, and recovery does not erase evidence. A kill switch can block new entries while exit/reconciliation behavior follows a separately defined policy; test that distinction.

Repeated reconnects can leak sockets, tasks or file descriptors. A soak test records resource counts over many cycles and checks they return toward a stable baseline. A process being active does not mean useful work is progressing. Monitor last successful tick, queue lag, oldest unresolved order, reconciliation failures and per-service heartbeats.

Safe operational inspection

Read-only commands on an authorized staging machine can include `systemctl is-active`, `recent journalctl --no-pager` entries, disk usage and memory usage. Redact credentials before sharing logs. A Redis authentication error is different from Redis being slow; investigate the connection identity and active configuration rather than immediately increasing machine size.

Practice and checkpoint

Design a recovery test where a fill arrives after a network timeout. The system should reconcile the existing intent before retrying. State how you would prove there was no duplicate submission and how long unresolved state may remain before operator intervention is required.

Chapter 39. Full-Project Feature Matrix and Capstone

Convert feature lists into evidence

The table below is a **test-design inventory**, not a claim that every row is implemented in the companion lab or verified against a live broker. Map it to the current production code and record actual results. The lab provides a safe foundation; advanced features are capstone extensions.

Feature	Essential positive case	Essential failure or invariant
Admin setup	Allowed identity claims once	Concurrent or unauthorized claim rejected
Password/TOTP	Valid two-step login	Expiry, replay policy, logout, throttling
Broker credentials	New credentials reach workers	Old client not reused after token change
Webhook URL	Correct configured origin	Old VM origin warns; secret not leaked
Alert names	Intended normalization matches	Collision and missing config explicit
Sizing	Fixed and capital policy	Zero, negative, expensive stock policy
Entry window	Boundary-time acceptance	After end blocked, timezone fixed
Trade limits	Allowed reservation	Concurrent entries cannot exceed policy
Sector cache	Valid previous session close	Zero denominator and stale cache rejected
Sector ranking	LONG gainers, SHORT losers	Unknown sector and stale ranking explicit
Breakout	Exact previous closed bar	No forming bar substitution
Breakout TTL	Cross within original TTL	Retry/restart cannot extend deadline
Tick stream	Resolve, subscribe, consume	Missing ID, stale/out-of-order tick
Fallback	Labelled REST price	Must not claim WebSocket healthy
Entry execution	Confirm actual fills	Acceptance alone never full position
Partial fills	Position uses filled quantity	Retry only reconciled remainder
Limit price	Side-aware tick alignment	Circuit/funds/unknown rejection handling
Stop and target	BUY and SELL exact boundary	No exit from unrelated strategy
Trailing stop	Favourable movement updates	BUY stop never decreases
Cost stop	Trigger at configured RR	Disabled toggle prevents application
Pyramiding	Confirm add, update average	Failed add does not inflate quantity
CNC carry	Restore owned carry records	Empty holdings alone not proof of sale
Auto square-off	Intended product/time scope	CNC policy not silently treated as MIS
Daily P&L	Threshold and scope correct	App kill and broker kill distinguished

Feature	Essential positive case	Essential failure or invariant
Alert exits	Matching owner and symbol	Other strategy unaffected
Reconciliation	Missing updates recovered	External/manual trades not misattributed
Telegram	Enabled strategy emits once	Disabled, token failure, duplicate message
Backtest	Deterministic known candles	CPU task does not block live event loop
Dashboard	Correct row identity and P&L	Skipped/rejected row cannot become LIVE
Restart	State/ownership restored	Duplicate workers cannot double-submit

A staged capstone

Stage 1: write a manual test plan, ten defects seeded in your disposable fork, and a traceability matrix. Stage 2: implement domain tests for targets, stops, tick sizes and sizing. Stage 3: add API validation and idempotency cases. Stage 4: build browser configuration and position flows. Stage 5: add a fake delayed broker, partial fills, a fake clock and a durable store. Stage 6: run a CI pipeline and publish a redacted report with limitations.

Your deliverables are a runnable repository, architecture diagram, test plan, automated tests, sample failure trace, defect reports, CI evidence and release recommendation. Demonstrate one bug by making the regression test fail before the fix and pass after it. Do not submit screenshots of green badges without the code that produced them.

Review criteria

Reviewers should be able to set up the project from a clean machine, understand its safety boundary, run unit/API tests without broker credentials, and reproduce a browser scenario. They should see why each critical expected result is correct. They should also see what remains untested. Senior judgment includes declining unsupported safety claims.

Practice and checkpoint

Choose five rows from the matrix and assign each a unit test, integration test and manual observation. For at least one feature, explain why a browser-only test is insufficient. For reconciliation, the answer should include broker evidence and durable state rather than only a dashboard label.

Chapter 40. From Beginner to SDET: Study Plan and Interview Practice

A realistic learning path

An SDET combines programming, test design, debugging, system understanding and delivery engineering. Completing a course or book helps, but job readiness comes from independently solving problems and explaining tradeoffs. No book can guarantee a job title, salary or seniority. Use these milestones as evidence of growth, not a calendar promise.

Stage	Suggested focus	Exit evidence
Weeks 1-2	Manual fundamentals and risk	Test plan, boundaries, five clear defects
Weeks 3-5	Python and small algorithms	Own calculations with edge-case tests
Weeks 6-7	Pytest and API contracts	Isolated fixtures, negative cases, coverage review
Weeks 8-10	Playwright and framework	Reliable locators, tracing, mobile flows
Weeks 11-12	CI, Git, reporting	Pipeline catches a seeded failure
Weeks 13-16	Distributed failures and capstone	Reconciliation/race tests and release report

Study time varies. Repeat a stage until you can explain and implement its evidence without copying. Spend roughly as much time debugging and designing tests as watching lessons. After each chapter, close the book and rebuild the example from its requirement. Compare afterward and explain differences.

Coding skills for interviews

Practice strings, dictionaries, sets, sorting, searching, stacks, queues and complexity analysis. Useful project exercises include deduplicating alerts while preserving order, merging order updates by identity, detecting missing sequence numbers and finding top-N sectors. Explain why a dictionary lookup is typically efficient and why retaining an unbounded event list is a memory risk. Do not memorize algorithms without connecting them to data contracts.

System-design discussion

Draw client -> API -> queue -> workers -> store -> broker, plus the return path for fills and ticks. Discuss ownership, idempotency, retries, observability and failure recovery before choosing many microservices. Explain which state must survive restart and what can be recomputed. Describe how you would test each boundary and how a test environment differs from production.

Interview prompts: How do you prevent duplicate orders after a timeout? How do you isolate parallel tests? Why can a 200 response be semantically wrong? How do you test a 9:00 IST job without waiting until tomorrow? How do you distinguish a flaky assertion from a real race? How do you test authentication without storing real credentials?

Answer model

For duplicate orders, say: record a durable intent/correlation identity, treat timeout as unknown, query broker evidence, apply confirmed fills idempotently, and only retry a known remaining quantity under ownership control. Then describe the test: accept remotely, drop the response, restart the worker, redeliver the same event, and assert no second full order. This is much stronger than saying simply to add three retries.

Final checkpoint

Can another beginner reproduce your tests? Can you explain every assertion? Can you diagnose a deliberately broken dependency? Can you state the limits of your evidence without becoming defensive? If yes, you are developing the habits expected of an effective SDET. Continue with production-like staging exercises, peer review and deeper database/network/security study rather than treating the final page as the end of learning.

Part VII. Workbook and Reference

Appendix A. Manual Test Pack You Can Execute and Extend

These cases are requirements-based templates. Cases marked **Lab** are supported by the companion application. Cases marked **Extension** require you to add a fake implementation or use an isolated copy of the production app. Do not report extension cases as executed merely because this table exists. For each run record commit, test data, actual result and evidence.

ID / scope	Setup and action	Expected observation
M01 Lab	Fresh app; GET strategies	Empty list, successful response
M02 Lab	Save alpha qty 2 through UI	Success message; API returns qty 2
M03 Lab	Save qty 0 through direct API	Validation error; no new config
M04 Lab	Save qty true through API	Strict integer validation rejects
M05 Lab	Signal before config exists	CFG_MISSING; zero positions
M06 Lab	Save alpha then signal TEST at 100	OPEN qty 2, entry 100
M07 Lab	Repeat same event and same payload	Same response, no extra position
M08 Lab	Same event ID, different price	Conflict, no second submission
M09 Lab	New event, same open strategy/symbol	ALREADY_OPEN conflict
M10 Lab	BUY tick at 100.99	Remains OPEN, P&L 1.98
M11 Lab	BUY tick at 101	CLOSED TARGET, P&L 2.00
M12 Lab	BUY tick at 99.50	CLOSED STOP_LOSS, P&L -1.00
M13 Lab	SELL tick at 99	CLOSED TARGET, P&L 2.00
M14 Lab	SELL tick at 100.50	CLOSED STOP_LOSS, P&L -1.00
M15 Lab	Tick after position CLOSED	Closed P&L unchanged
M16 Lab	Alpha/beta own TEST; exit alpha	Beta still OPEN
M17 Lab	Delete config then list	Config absent; existing position not erased
M18 Lab	Wrong symbol characters	Validation error, no position
M19 Lab	Mobile 390 px, complete form	Save reachable, no document overflow
M20 Lab	Mock positions API 502 JSON	Error visible; not false success
M21 Extension	Save retries 0 and buffer 0	Both preserved after reload
M22 Extension	TSL off, favourable tick	No trailing update
M23 Extension	BUY TSL moves up then price falls	TSL does not move back down

ID / scope	Setup and action	Expected observation
M24 Extension	Cost SL off; RR reached	No cost-SL action
M25 Extension	SL .6%, RR 2, BUY rises 1.2%	Cost rule activates per boundary policy
M26 Extension	SHORT sector top-N enabled	Most negative sectors selected first
M27 Extension	Missing previous sector close	Ranking not fabricated as zero change
M28 Extension	Missing candle then late valid candle	Fixed reference, original TTL retained
M29 Extension	Breakout after TTL	No order
M30 Extension	Duplicate watch after restart	No duplicate watch/order or TTL extension
M31 Extension	Broker PENDING	No full filled position
M32 Extension	Fill 4 of requested 10	Owned quantity 4; remainder 6
M33 Extension	Insufficient funds rejection	Reason visible; no false LIVE entry
M34 Extension	Lost broker response after acceptance	Reconcile before possible replacement
M35 Extension	Feed connects but no symbol tick	Not represented as fresh data
M36 Extension	Old token replaced	Relevant workers use new credentials
M37 Extension	Old skip plus new same-symbol trade	Skip remains skip; only correct trade LIVE
M38 Extension	API request without admin session	Protected data unavailable
M39 Extension	Two simultaneous admin setup requests	One owner only
M40 Extension	Logout then reuse session	Sensitive access denied per policy
M41 Extension	TOTP reused in same step	Enforced documented replay policy
M42 Extension	Wrong webhook secret	No queued trade candidate
M43 Extension	Alert exit for wrong strategy	No unrelated position closed
M44 Extension	CNC next-day restore, holdings delayed	Preserve unresolved ownership record
M45 Extension	Telegram toggle off	No notification sent
M46 Extension	Duplicate exit and target tick together	At most owned remaining quantity exits
M47 Extension	Worker crashes after claim	Work recovered, not duplicated blindly
M48 Extension	Backtest runs during tick flow	Live handling remains within measured budget

Reusable defect template

TEXT

```
ID:  
Title: observable symptom, not a guessed cause  
Requirement / risk:  
Build and environment:  
Preconditions and synthetic data:  
Minimal steps:  
Expected:  
Actual:  
Reproducibility:  
Severity and proposed priority:  
Redacted evidence / correlation IDs:  
Workaround and its risks:  
Fix verification and regression test:
```

Example: "Historical SKIPPED alert becomes LIVE after later same-symbol entry." Expected: immutable alert outcome remains SKIPPED; a separately identified trade row is LIVE. Risk: operator may close or interpret the wrong event. Verification requires at least two alert episodes and a matching symbol, not a single happy-path position.

Appendix B. Worked Solutions and Debugging Drills

Drill 1: the zero-default bug

Faulty expression: `retries = payload.get('retries') or 1`. Trace it with input 0: lookup returns 0, Python considers it false, so `or` selects 1. Fix the policy first: is null allowed, and does omission mean default? For a strict API, reject null or handle it explicitly; use `.get('retries', 1)` when only omission should default. Test absent, zero, one, negative, string and null separately. Add a browser save/reload test to catch serialization mistakes.

Drill 2: false confidence from a green badge

A status response says `authenticated=true`, `dashboard socket connected=true`, but no symbol has a recent broker receive timestamp. The correct conclusion is partial health, not full data readiness. Your evidence checklist is authentication, broker transport state, subscription identity, latest source/age, risk-consumer receipt and browser receipt. The UI should present distinct indicators. Do not substitute a current REST timestamp into a WebSocket-health field.

Drill 3: target arithmetic and display

Entry 427.50, target 1% gives 431.775 before display rounding. Two-decimal half-up display is 431.78. Stop 0.5% gives 425.3625, displayed 425.36. A configured trailing distance of 0.8% initially gives 424.08 if calculated directly from entry, but an engine may use the tighter protective stop as the effective line. Test the documented effective-stop policy; do not assume a displayed TSL must always equal entry minus the trailing percentage. Order tick alignment is yet another separate rule.

Drill 4: the delayed breakout candle

At 10:30:03 IST with one-minute breakout, reference the closed 10:29-10:30 bar. If its high is 100 and the absolute buffer is 0.10, the threshold is 100.10. If the policy is strict break, 100.10 does not cross; 100.11 does. A two-minute watch retains its original expiry despite repeated fetches. If the candle appears late, use the original reference bar, not the latest forming candle. Buffer units must be specified: an absolute 0.10 is not the same as 0.10%.

Drill 5: partial fill and retry

Requested 10; confirmed fill 4; cancel requested; final broker reconciliation says fill 6 before cancellation. The known remaining amount is now 4, not 6. A retry based on the earlier snapshot can overbuy. Build a scripted fake that returns these exact stages and assert the next submitted quantity. If the final outcome is unknown, do not guess a remainder. Preserve unresolved state and escalate according to policy.

Drill 6: flaky concurrency assertion

The test starts a 150 ms fake data call, then demands an order finish within 50 ms. On a loaded machine, scheduling alone can violate the number. To prove independence, hold the data call behind an event and assert the order completes before releasing it. To prove latency, run a separate benchmark with defined hardware, sample count, load and percentile target. Combining the two questions produces confusing evidence.

Drill 7: wrong alert name

Incoming alert is high volume stock; saved strategy is Alert for F&O INTRADAY BULLISH SCANNER. These are different names. Redis health cannot make them the same configuration. Inspect the received normalized key, user scope and saved key. Test intended normalization and collision rejection. Do not implement a fallback to the first enabled strategy: that would route an unknown signal into unrelated trading rules.

Drill 8: HTTP success versus business success

HTTP transport succeeds, but the response body is an SDK failure envelope with an empty data object. Treating it as an empty candle list hides the broker failure. The adapter should distinguish valid empty data, malformed structure and explicit failure. Test each. User-facing messages should be safe and actionable, while redacted logs retain diagnostic categories.

Drill 9: temporal ownership

An old entry and a new entry share strategy and symbol but have different trade IDs. A dashboard joins only on symbol and attaches new P&L to the old row. Add trade-episode identity to the join. A regression test should vary both strategy and trade ID independently. An ownership rule is not complete if it handles two strategies but still confuses two trades from the same strategy.

Drill 10: ambiguous deployment failure

Nginx returns 502; systemd reports API active; the API port listens. These facts do not prove successful request handling. Check a local read-only health endpoint and application traceback, then reverse-proxy upstream errors. Correlate times, process IDs and recent changes. Avoid sending a real webhook as a diagnostic because it can create an order if the system recovers during the test.

Appendix C. Syllabus Coverage and Additional Depth

Requested topic	Where to study and practice
Manual testing, SDLC/STLC, UAT, defects	Chapters 1-5; Appendix A
Setup, comments, variables and operators	Chapters 6-7
Strings, lists, tuples, sets, dictionaries	Chapters 7-8
Conditions, loops, formatted output	Chapters 7-8
Functions, scope, modules and packages	Chapter 9
OOP, inheritance, polymorphism, abstraction	Chapter 10
File handling and data-driven inputs	Chapters 11, 16 and 30
Exceptions and advanced Python	Chapters 12-13
Pytest, fixtures, marks and parameterization	Chapters 14-18
Playwright setup and built-in locators	Chapters 19-20
CSS, XPath, Shadow DOM, SelectorsHub	Chapter 20
Inputs, radio, checkbox, native/custom menus	Chapter 21
Static/dynamic/paginated tables	Chapter 22
jQuery/Bootstrap calendars, dialogs, frames	Chapter 23
Mouse, keyboard, upload and download	Chapter 24
Contexts, tabs, popups and auth state	Chapter 25
Screenshots, video, traces and flaky tests	Chapter 26
Request mocking and stream evidence	Chapter 27
Cross-browser, mobile and parallel testing	Chapter 28
Page Object Model, framework development	Chapters 29-30
Pytest and Allure reporting	Chapter 30
REST methods and CRUD	Chapter 31
Headers, cookies and JSON Schema	Chapter 32
Basic/Bearer/API-key/OAuth, email and TOTP	Chapter 33

Requested topic	Where to study and practice
Full-stack integration and security	Chapter 34
Git, GitHub, Jenkins and GitHub Actions	Chapters 35-36
Codegen, MCP, PyCharm and Copilot	Chapter 37
Load, resilience, queues and observability	Chapters 18 and 38
Project capstone and SDET preparation	Chapters 39-40

Beyond the core: what to study next

Property-based testing generates many values from a stated invariant. For example, valid tick rounding should return a multiple of the tick and must not round a BUY below the input price. Use a library such as Hypothesis in a separate extension exercise; shrinking a failure to a minimal example is often more useful than retaining a giant random dataset.

Mutation testing deliberately changes small operators or conditions and checks whether tests notice. Surviving mutations can expose weak assertions, but some mutations are equivalent or outside meaningful requirements. Review results instead of chasing a single score.

Consumer/provider contract testing checks that services agree on schemas and semantics without running an entire system for every change. A schema-only check is insufficient for order state transitions. Version both response shapes and state assumptions.

Accessibility, security and performance need specialist depth. Treat browser checks, a security checklist and a small load run as beginnings, not certification. For senior roles, also study SQL query plans, networking, Linux processes, containers, cloud IAM, observability, architecture decisions and incident response. The existing engineering book in this repository develops cloud and Kubernetes separately; this volume concentrates on testing.

Appendix D. Glossary and Release Report

Term	Practical meaning
Assertion	Check comparing observed behavior with an expectation
Oracle	Independent source of expected behavior
Fixture	Controlled setup/data/resource supplied to a test
Stub	Substitute returning predefined responses
Fake	Simplified working implementation of a dependency
Spy	Recorder of calls or interactions
Contract	Agreed input, output and semantic behavior at a boundary
Idempotency	Repeating an operation does not create extra intended effects
Reconciliation	Comparing local records with authoritative evidence
Partial fill	Only part of requested quantity has executed
Race	Outcome depends on interleaving of concurrent operations
Lease	Time-bounded ownership requiring renewal
TTL	Lifetime after which data or work expires
Regression	Previously supported behavior broken by a change
Flaky test	Outcome varies without an intended relevant change
Smoke test	Small check that a build/environment is usable
UAT	Business-user evaluation against acceptance needs
SDET	Engineer combining software development and test engineering
P95 latency	Value at or below which roughly 95% of sampled durations lie
Test double	General term for a dependency substitute
DOM	Browser representation of document structure
Context	Isolated browser-session boundary in Playwright
Trace	Recorded execution evidence for debugging
CI	Automated integration checks for proposed changes
Risk-based testing	Prioritizing evidence by impact and likelihood

Release summary template

TEXT

```
Build / commit:
Environment and dependency versions:
Requirements in scope:
Requirements excluded:
Tests collected / passed / failed / skipped / expected-failed:
Browser engines and viewport sizes actually executed:
Broker mode: fake, recorded fixture, sandbox, or authorized live check:
Critical invariants checked:
Unresolved defects and mitigations:
Performance workload and results, if measured:
Artifact locations, access controls and retention:
Recommendation: release / restricted release / hold:
Decision owner and date:
```

A truthful report may recommend holding a release even when most tests pass. Conversely, a known cosmetic issue may be acceptable with an explicit decision. The tester provides evidence and risk analysis; do not conceal failures to make a dashboard green.

Appendix E. Primary Sources and Reading Order

Documentation was consulted while preparing this edition. APIs, IDE menus and installation requirements can change; use these primary sources alongside the dependency versions recorded in your own environment. Explanations and exercises in this book are original teaching material, not copied course transcripts.

- [Python tutorial](#): language reference path for Parts II-III.
- [Pytest fixtures](#): fixture lifecycle and scope.
- [Playwright installation](#): setup and browser installation.
- [Playwright locators](#): supported locator behavior.
- [Playwright actions](#): form and input APIs.
- [Playwright dialogs](#): browser-dialog lifecycle.
- [Playwright frames](#): iframe interaction.
- [Playwright downloads](#): download events and saving.
- [Playwright authentication](#): session-state reuse precautions.
- [Playwright API testing](#): APIRequestContext.
- [Playwright network](#): interception and observation.
- [Playwright pytest plugin](#): execution options.
- [Playwright trace viewer](#): diagnostic traces.
- [Playwright Codegen](#): recording workflows.
- [Playwright MCP](#): official browser MCP server.
- [JetBrains MCP client](#): IDE-client setup.
- [GitHub Copilot MCP](#): client integration.
- [GitHub Python workflows](#): CI concepts.
- [Jenkins pipeline syntax](#): pipeline configuration.
- [Allure pytest](#): report integration.
- [OWASP testing guide](#): authorized security assessment.
- [Dhan order contract](#): broker order semantics.
- [Kite order contract](#): broker order semantics.

Start with concepts, implement the lab, then read the corresponding production tests. Do not begin by executing client-account operations. The goal is to understand and prove software behavior in a controlled environment before approaching operational risk.

Appendix F. Executable Lab Source and Walkthrough

The following files form the companion lab. Begin with `domain.py`, then follow `create_app` in `server.py`. Each call builds independent state. API tests use `TestClient`; browser tests use a short-lived localhost server and fresh contexts. The DOM-control tests create their own HTML instead of relying on public demo sites. The memory dictionaries are deliberately not a durable multi-process database.

requirements.txt

TEXT

```
fastapi>=0.115,<1
uvicorn>=0.30,<1
pydantic>=2,<3
httpx>=0.27,<1
pytest>=8,<10
pytest-playwright>=0.5,<1
pytest-xdist>=3,<4
jsonschema>=4,<5
```

pytest.ini

INI

```
[pytest]
testpaths = tests
addopts = -ra --strict-markers
markers =
    api: in-process API integration tests
    browser: local browser integration tests
```

domain.py

PYTHON

```
"""Paper-only teaching model. No brokerage or network dependencies."""
from dataclasses import dataclass
from decimal import Decimal, ROUND_CEILING, ROUND_FLOOR

def positive(value):
    number = Decimal(str(value))
    if not number.is_finite() or number <= 0:
        raise ValueError("Expected a finite positive value")
    return number

def tick_price(price, tick, side):
    price, tick = positive(price), positive(tick)
    if side not in {"BUY", "SELL"}:
        raise ValueError("Invalid side")
    mode = ROUND_CEILING if side == "BUY" else ROUND_FLOOR
    result = (price / tick).to_integral_value(rounding=mode) * tick
    return positive(result)

def quantity(capital, price, minimum_one=False):
    capital, price = positive(capital), positive(price)
    result = int(capital // price)
    return max(1, result) if minimum_one else result

def sector_change(ltp, previous_close):
    return (positive(ltp) / positive(previous_close) - 1) * 100

@dataclass
```

```

class Position:
    strategy: str
    symbol: str
    side: str
    qty: int
    entry: Decimal
    target: Decimal
    stop: Decimal
    ltp: Decimal
    status: str = "OPEN"
    exit_reason: str = ""

    @classmethod
    def from_fill(cls, strategy, symbol, side, qty, price, target_pct, sl_pct):
        entry = positive(price)
        target_pct, sl_pct = positive(target_pct), positive(sl_pct)
        if type(qty) is not int or qty <= 0 or side not in {"BUY", "SELL"}:
            raise ValueError("Invalid fill")
        if target_pct >= 100 or sl_pct >= 100:
            raise ValueError("Percentages must be below 100")
        sign = 1 if side == "BUY" else -1
        return cls(strategy, symbol, side, qty, entry,
                    entry * (1 + sign * target_pct / 100),
                    entry * (1 - sign * sl_pct / 100), entry)

    @property
    def pnl(self):
        sign = 1 if self.side == "BUY" else -1
        return (self.ltp - self.entry) * self.qty * sign

    def mark(self, price):
        if self.status != "OPEN":
            return
        self.ltp = positive(price)
        target_hit = self.ltp >= self.target if self.side == "BUY" else self.ltp <= self.target
        stop_hit = self.ltp <= self.stop if self.side == "BUY" else self.ltp >= self.stop
        if target_hit or stop_hit:
            # A lab assumes a fill at the supplied tick, not a live execution promise.
            self.status = "CLOSED"
            self.exit_reason = "TARGET" if target_hit else "STOP_LOSS"

    def as_dict(self):
        return {"strategy": self.strategy, "symbol": self.symbol,
                "side": self.side, "qty": self.qty, "status": self.status,
                "entry": str(self.entry), "ltp": str(self.ltp),
                "target": str(self.target), "stop": str(self.stop),
                "pnl": str(self.pnl), "exit_reason": self.exit_reason}

```

server.py

PYTHON

```

"""Local teaching server: never deploy publicly; memory resets on restart."""
from decimal import Decimal
from pathlib import Path
from typing import Literal

from fastapi import FastAPI, HTTPException
from fastapi.responses import HTMLResponse
from pydantic import BaseModel, Field, ConfigDict

from domain import Position

class Strategy(BaseModel):
    model_config = ConfigDict(extra="forbid")
    name: str = Field(min_length=1, max_length=50, pattern=r"^[A-Za-z0-9 _-]+$")
    side: Literal["BUY", "SELL"] = "BUY"
    qty: int = Field(default=1, gt=0, le=100, strict=True)
    target_pct: Decimal = Field(default=Decimal("1"), gt=0, lt=100, allow_inf_nan=False)
    sl_pct: Decimal = Field(default=Decimal("0.5"), gt=0, lt=100, allow_inf_nan=False)

```

```

class Signal(BaseModel):
    model_config = ConfigDict(extra="forbid")
    event_id: str = Field(min_length=1, max_length=100)
    strategy: str
    symbol: str = Field(pattern=r"^[A-Z][A-Z0-9]{0,19}$")
    price: Decimal = Field(gt=0, allow_inf_nan=False)

class Tick(BaseModel):
    price: Decimal = Field(gt=0, allow_inf_nan=False)

def create_app():
    app = FastAPI(title="Paper Testing Lab")
    configs, positions, seen = {}, {}, {}

    @app.get("/", response_class=HTMLResponse)
    async def home():
        return Path(__file__).with_name("index.html").read_text(encoding="utf-8")

    @app.get("/health")
    async def health():
        return {"ok": True, "mode": "PAPER_ONLY"}

    @app.post("/api/strategies", status_code=201)
    async def save(config: Strategy):
        configs[config.name] = config
        return config

    @app.get("/api/strategies")
    async def list_configs():
        return list(configs.values())

    @app.delete("/api/strategies/{name}", status_code=204)
    async def delete(name: str):
        if name not in configs:
            raise HTTPException(404, "CONFIG_NOT_FOUND")
        del configs[name]

    @app.post("/api/signals")
    async def signal(data: Signal):
        payload = data.model_dump()
        if data.event_id in seen:
            old, result = seen[data.event_id]
            if old != payload:
                raise HTTPException(409, "IDEMPOTENCY_CONFLICT")
            return result
        if data.strategy not in configs:
            raise HTTPException(404, "CFG_MISSING")
        key = (data.strategy, data.symbol)
        if key in positions and positions[key].status == "OPEN":
            raise HTTPException(409, "ALREADY_OPEN")
        config = configs[data.strategy]
        position = Position.from_fill(config.name, data.symbol, config.side,
                                     config.qty, data.price, config.target_pct, config.sl_pct)
        positions[key] = position
        result = position.as_dict()
        seen[data.event_id] = (payload, result)
        return result

    @app.get("/api/positions")
    async def list_positions():
        return [p.as_dict() for p in positions.values()]

    @app.post("/api/ticks/{symbol}")
    async def tick(symbol: str, data: Tick):
        for position in positions.values():
            if position.symbol == symbol:
                position.mark(data.price)

```

```

    return {"ok": True}

@app.post("/api/exits/{strategy}/{symbol}")
async def exit_signal(strategy: str, symbol: str):
    position = positions.get((strategy, symbol))
    if position is None:
        raise HTTPException(404, "POSITION_NOT_FOUND")
    if position.status == "OPEN":
        position.status, position.exit_reason = "CLOSED", "ALERT_EXIT"
    return position.as_dict()

return app

app = create_app()

```

index.html

HTML

```

<!doctype html>
<html lang="en"><head><meta charset="utf-8"><meta name="viewport" content="width=device-width,initial-scale=1">
<title>Paper Testing Lab</title><style>
body{font:16px system-ui;margin:0;color:#20342e;background:#f5f8f7}main{max-width:1000px;margin:auto;padding:24px}
form{display:grid;grid-template-columns:repeat(3,minmax(0,1fr));gap:14px;padding:20px 0;border-block:1px solid
#c8d8d0}
label{display:grid;gap:5px}input,select,button{font:inherit;padding:10px;border:1px solid #91a89c;border-
radius:4px;min-width:0}
button{background:#16684d;color:white;cursor:pointer}table{border-
collapse:collapse;width:100%;td,th{padding:10px;border-bottom:1px solid #c8d8d0;text-align:left}
.scroll{overflow:auto;}#message{min-height:24px} @media(max-width:600px){form{grid-template-
columns:1fr}main{padding:16px}}
</style></head><body><main><h1>Paper Testing Lab</h1><p>No broker connection. Synthetic trades only.</p>
<form id="config"><label>Strategy name<input id="name" value="demo" required></label>
<label>Direction<select id="side"><option>BUY</option><option>SELL</option></select></label>
<label>Quantity<input id="qty" type="number" min="1" max="100" value="2" required></label>
<label>Target percent<input id="target" type="number" step="0.1" min="0.1" value="1" required></label>
<label>Stop percent<input id="stop" type="number" step="0.1" min="0.1" value="0.5" required></label>
<button>Save strategy</button></form><p id="message" role="status"></p>
<button id="refresh">Refresh positions</button><div class="scroll"><table aria-label="Positions"><thead><tr>
<th>Strategy</th><th>Symbol</th><th>Quantity</th><th>Entry</th><th>LTP</th><th>P&L</th><th>Status</th><th>Reason<
/th>
</tr></thead><tbody id="rows"></tbody></table></div></main><script>
async function api(path, options={}) {
  const response = await fetch(path, options);
  const body = await response.json();
  if (!response.ok) throw new Error(typeof body.detail === 'string' ? body.detail : 'Validation failed');
  return body;
}
async function refresh() {
  try {
    const data = await api('/api/positions');
    const rows = document.getElementById('rows'); rows.replaceChildren();
    for (const p of data) {
      const row = document.createElement('tr');
      for (const key of ['strategy', 'symbol', 'qty', 'entry', 'ltp', 'pnl', 'status', 'exit_reason']) {
        const cell = document.createElement('td'); cell.dataset.field=key;
        cell.textContent=['entry', 'ltp', 'pnl'].includes(key) ? Number(p[key]).toFixed(2) : p[key];
        row.append(cell);
      } rows.append(row);
    }
  } catch(error) { document.getElementById('message').textContent=error.message; }
}
document.getElementById('config').addEventListener('submit', async event => {
  event.preventDefault();
  const get = id => document.getElementById(id).value;
  try {
    await api('/api/strategies', {method:'POST', headers:{'Content-Type':'application/json'},
      body:JSON.stringify({name:get('name'),side:get('side'),qty:Number(get('qty')),
        target_pct:get('target'),sl_pct:get('stop')}}));
    document.getElementById('message').textContent='Strategy saved';
  }

```

```

    } catch(error) { document.getElementById('message').textContent=error.message; }
});
document.getElementById('refresh').addEventListener('click',refresh); refresh();
</script></body></html>

```

tests/conftest.py

PYTHON

```

import asyncio
import socket
import threading
import time

import pytest
import uvicorn
from fastapi.testclient import TestClient
from server import create_app

@pytest.fixture
def client():
    with TestClient(create_app()) as value:
        yield value

@pytest.fixture
def lab_url():
    sock = socket.socket()
    sock.bind(("127.0.0.1", 0))
    port = sock.getsockname()[1]
    server = uvicorn.Server(uvicorn.Config(create_app(), log_level="error"))
    thread = threading.Thread(target=lambda: asyncio.run(server.serve(sockets=[sock])))
    thread.start()
    try:
        deadline = time.monotonic() + 10
        while not server.started:
            if not thread.is_alive() or time.monotonic() > deadline:
                raise RuntimeError("Lab server did not start")
            time.sleep(.02)
        yield f"http://127.0.0.1:{port}"
    finally:
        server.should_exit = True
        thread.join(10)
        sock.close()
        assert not thread.is_alive(), "Lab server failed to stop"

```

tests/test_domain.py

PYTHON

```

from decimal import Decimal
import pytest
from domain import Position, quantity, sector_change, tick_price

@pytest.mark.parametrize("side,expected", [("BUY", "100.05"), ("SELL", "100.00")])
def test_directional_tick_rounding(side, expected):
    assert tick_price("100.03", ".05", side) == Decimal(expected)

@pytest.mark.parametrize("bad", ["0", "-1", "NaN", "Infinity"])
def test_invalid_price_rejected(bad):
    with pytest.raises(ValueError):
        quantity("10000", bad)

def test_capital_policy_is_explicit():
    assert quantity("10000", "12000") == 0
    assert quantity("10000", "12000", minimum_one=True) == 1

```

```
def test_sector_denominator_and_sign():
    assert sector_change("990", "1000") == Decimal("-1")

@pytest.mark.parametrize("side,tick,reason,pnl", [
    ("BUY", "101", "TARGET", "2"), ("BUY", "99.5", "STOP_LOSS", "-1"),
    ("SELL", "99", "TARGET", "2"), ("SELL", "100.5", "STOP_LOSS", "-1")])
def test_exit_at_exact_threshold(side, tick, reason, pnl):
    position = Position.from_fill("demo", "TEST", side, 2, "100", "1", ".5")
    position.mark(tick)
    assert position.status == "CLOSED"
    assert position.exit_reason == reason
    assert position.pnl == Decimal(pnl)
    position.mark("1000")
    assert position.pnl == Decimal(pnl)
```

tests/test_api.py

PYTHON

```
import pytest

pytestmark = pytest.mark.api

def seed(client, name="demo"):
    response = client.post("/api/strategies", json={"name": name, "qty": 2})
    assert response.status_code == 201

def payload(event="one", strategy="demo"):
    return {"event_id": event, "strategy": strategy, "symbol": "TEST", "price": "100"}

def test_config_lifecycle(client):
    seed(client)
    assert client.get("/api/strategies").json()[0]["qty"] == 2
    assert client.delete("/api/strategies/demo").status_code == 204
    assert client.get("/api/strategies").json() == []

def test_config_missing_has_no_position(client):
    response = client.post("/api/signals", json=payload())
    assert response.status_code == 404
    assert response.json()["detail"] == "CFG_MISSING"
    assert client.get("/api/positions").json() == []

@pytest.mark.parametrize("qty", [0, -1, True, "2", 101])
def test_bad_qty_has_no_write(client, qty):
    assert client.post("/api/strategies", json={"name": "bad", "qty": qty}).status_code == 422
    assert client.get("/api/strategies").json() == []

def test_duplicate_and_conflicting_signal(client):
    seed(client)
    first = client.post("/api/signals", json=payload())
    assert first.status_code == 200
    assert client.post("/api/signals", json=payload()).json() == first.json()
    assert len(client.get("/api/positions").json()) == 1
    changed = dict(payload(), price="101")
    assert client.post("/api/signals", json=changed).status_code == 409

def test_exit_cannot_close_other_strategy(client):
    for name in ("alpha", "beta"):
        seed(client, name)
        assert client.post("/api/signals", json=payload(name, name)).status_code == 200
        client.post("/api/exits/alpha/TEST")
```

```

states = {p["strategy"]: p["status"] for p in client.get("/api/positions").json()}
assert states == {"alpha": "CLOSED", "beta": "OPEN"}

def test_tick_target_and_closed_pnl(client):
    seed(client)
    client.post("/api/signals", json=payload())
    assert client.post("/api/ticks/TEST", json={"price": "101"}).status_code == 200
    position = client.get("/api/positions").json()[0]
    assert position["exit_reason"] == "TARGET"
    assert position["pnl"] == "2"

```

tests/test_browser.py

PYTHON

```

import pytest
from playwright.sync_api import expect

pytestmark = pytest.mark.browser

@pytest.mark.parametrize("width,height", [(1280, 900), (390, 844)])
def test_ui_api_position_lifecycle(page, lab_url, width, height):
    page.set_viewport_size({"width": width, "height": height})
    page.goto(lab_url)
    page.get_by_label("Strategy name").fill("student")
    page.get_by_label("Quantity").fill("2")
    page.get_by_role("button", name="Save strategy").click()
    expect(page.get_by_role("status")).to_have_text("Strategy saved")
    configs = page.request.get(lab_url + "/api/strategies").json()
    assert configs[0]["name"] == "student" and configs[0]["qty"] == 2
    response = page.request.post(lab_url + "/api/signals", data={
        "event_id": "browser-one", "strategy": "student", "symbol": "TEST", "price": "100"})
    assert response.ok
    page.get_by_role("button", name="Refresh positions").click()
    row = page.get_by_role("row").filter(has_text="TEST")
    expect(row.locator('[data-field="status"]')).to_have_text("OPEN")
    assert page.request.post(lab_url + "/api/ticks/TEST", data={"price": "101"}).ok
    page.get_by_role("button", name="Refresh positions").click()
    expect(row.locator('[data-field="status"]')).to_have_text("CLOSED")
    expect(row.locator('[data-field="pnl"]')).to_have_text("2.00")
    expect(row.locator('[data-field="exit_reason"]')).to_have_text("TARGET")
    assert page.evaluate("document.documentElement.scrollWidth <= innerWidth")

```

tests/test_controls.py

PYTHON

```

"""Self-contained DOM exercises; no external website is contacted."""
import csv
import pytest
from playwright.sync_api import expect

pytestmark = pytest.mark.browser

def test_inputs_radios_select(page):
    page.set_content('''<label>Capital<input type="checkbox"></label>
<label><input type="checkbox">Trailing SL</label>
<label><input type="radio" name="side">Long</label>
<label>Product<select><option value="MIS">Intraday</option>
<option value="CNC">Delivery</option></select></label>''')
    page.get_by_label("Capital").fill("5000")
    page.get_by_label("Trailing SL").check()
    page.get_by_label("Long", exact=True).check()
    page.get_by_label("Product").select_option("CNC")
    expect(page.get_by_label("Capital")).to_have_value("5000")
    expect(page.get_by_label("Trailing SL")).to_be_checked()
    expect(page.get_by_label("Long", exact=True)).to_be_checked()

```

```

expect(page.get_by_label("Product")).to_have_value("CNC")

def test_css_xpath_and_row_scope(page):
    page.set_content('<table><tr data-trade="one"><td>TEST</td><td>2.00</td></tr>
    <tr data-trade="two"><td>TEST</td><td>-1.00</td></tr></table>')
    expect(page.locator('[data-trade="one"]').locator('td').nth(1)).to_have_text("2.00")
    expect(page.locator("xpath=//tr[@data-trade='two']/td[2]")).to_have_text("-1.00")

def test_open_shadow_root(page):
    page.set_content('<div id="host"></div>')
    page.evaluate('`() => {
        document.querySelector('#host').attachShadow({mode: 'open'}).innerHTML =
        '<button>Reconnect feed</button>';
    }')
    expect(page.get_by_role("button", name="Reconnect feed")).to_be_visible()

def test_native_date(page):
    page.set_content('<label>From date<input type="date"></label>')
    page.get_by_label("From date").fill("2026-09-23")
    expect(page.get_by_label("From date")).to_have_value("2026-09-23")

@pytest.mark.parametrize("accept,expected", [(True, "true"), (False, "false")])
def test_dialog_both_outcomes(page, accept, expected):
    page.set_content('<button onclick="document.body.dataset.ok=confirm('Close paper?')">
    Close paper</button>')
    page.once("dialog", lambda dialog: dialog.accept() if accept else dialog.dismiss())
    page.get_by_role("button", name="Close paper", exact=True).click()
    expect(page.locator("body")).to_have_attribute("data-ok", expected)

def test_frame(page):
    page.set_content('<iframe title="Paper help"
    srcdoc="<button>Read risk notice</button>"></iframe>')
    expect(page.frame_locator('iframe[title="Paper help"]').get_by_role("button")).to_have_text("Read risk notice")

def test_upload(page, tmp_path):
    path = tmp_path / "symbols.csv"
    path.write_text("symbol,qty\nTEST,2\n", encoding="utf-8")
    page.set_content('<label>Import symbols<input type="file"></label>')
    control = page.get_by_label("Import symbols")
    control.set_input_files(path)
    assert control.evaluate("el => el.files.length") == 1
    assert control.evaluate("el => el.files[0].name") == "symbols.csv"

def test_download_content(page, tmp_path):
    page.set_content('<a download="positions.csv">Export CSV</a>')
    page.locator('a').evaluate(r`() => {
        el.href = URL.createObjectURL(new Blob(['symbol,qty\nTEST,2\n'], {type: 'text/csv'}));
    }')
    with page.expect_download() as pending:
        page.get_by_role("link", name="Export CSV").click()
    target = tmp_path / "positions.csv"
    pending.value.save_as(target)
    with target.open(newline="", encoding="utf-8") as stream:
        assert list(csv.DictReader(stream)) == [{"symbol": "TEST", "qty": "2"}]

def test_context_cookie_isolation(browser):
    first, second = browser.new_context(), browser.new_context()
    try:
        first.add_cookies([{"name": "paper", "value": "one", "url": "http://127.0.0.1"}])
        assert first.cookies()[0]["value"] == "one"
        assert second.cookies() == []
    finally:
        first.close()

```

```

        second.close()

def test_network_error_message(page, lab_url):
    page.route("**/api/positions", lambda route: route.fulfill(
        status=502, content_type="application/json", body='{"detail": "UPSTREAM_DOWN"}'))
    page.goto(lab_url)
    expect(page.get_by_role("status")).to_have_text("UPSTREAM_DOWN")

```

tests/test_widgets.py

PYTHON

```

"""Custom widgets are DOM contracts, not native select/date controls."""
import pytest
from playwright.sync_api import expect

pytestmark = pytest.mark.browser

def test_custom_dropdown(page):
    page.set_content('''
        <button onclick="document.querySelector('#menu').hidden=false">Choose product</button>
        <div role="listbox" aria-label="Products" id="menu" hidden>
            <button role="option" onclick="document.querySelector('#chosen').textContent='CNC';
                document.querySelector('#menu').hidden=true">Delivery</button>
        </div><output id="chosen" aria-label="Selected product"></output>
    ''')
    page.get_by_role("button", name="Choose product").click()
    menu = page.get_by_role("listbox", name="Products")
    menu.get_by_role("option", name="Delivery", exact=True).click()
    expect(menu).to_be_hidden()
    expect(page.locator('#chosen')).to_have_text("CNC")

def test_pagination_identity(page):
    page.set_content('''
        <table aria-label="Signals"><tbody id="rows"></tbody></table>
        <button id="next" onclick="render(++index)">Next</button>
        <script>
            let index=0; const records=['TEST', 'DEMO', 'PAPER'];
            function render(i){
                document.querySelector('#rows').innerHTML='<tr><td>'+records[i]+'</td></tr>';
                document.querySelector('#next').disabled=i===records.length-1;
            } render(index);
        </script>
    ''')
    names = []
    for expected in ["TEST", "DEMO", "PAPER"]:
        row = page.get_by_role("table", name="Signals").get_by_role("row")
        expect(row).to_have_text(expected)
        names.append(row.inner_text())
    if expected != "PAPER":
        page.get_by_role("button", name="Next").click()
    expect(page.get_by_role("button", name="Next")).to_be_disabled()
    assert names == ["TEST", "DEMO", "PAPER"]
    assert len(names) == len(set(names))

def test_custom_calendar_scopes_duplicate_day(page):
    page.set_content('''
        <section aria-label="August"><button>23</button></section>
        <section aria-label="September"><button
            onclick="document.querySelector('#date').value='2026-09-23'">23</button></section>
        <label>Chosen date<input id="date" readonly></label>
    ''')
    page.get_by_role("region", name="September").get_by_role("button", name="23", exact=True).click()
    expect(page.get_by_label("Chosen date")).to_have_value("2026-09-23")

def test_keyboard_and_tooltip(page):

```

```

page.set_content('''
    <button id="tool" onmouseenter="document.querySelector('#tip').hidden=false"
        onmouseleave="document.querySelector('#tip').hidden=true"
        onclick="document.querySelector('#result').textContent='requested'">Reconnect</button>
    <span id="tip" role="tooltip" hidden>Reconnect paper feed</span>
    <output id="result"></output>
''')
button = page.get_by_role("button", name="Reconnect", exact=True)
button.hover()
expect(page.get_by_role("tooltip")).to_have_text("Reconnect paper feed")
button.focus()
expect(button).to_be_focused()
button.press("Enter")
expect(page.locator('#result')).to_have_text("requested")

```

```

def test_popup_uses_returned_page(page):
    page.set_content('''<button onclick="window.open('about:blank')">Open paper help</button>''')
    with page.expect_popup() as pending:
        page.get_by_role("button", name="Open paper help").click()
    popup = pending.value
    popup.set_content('<h1>Risk guide</h1>')
    expect(popup.get_by_role("heading", name="Risk guide")).to_be_visible()
    popup.close()
    expect(page.get_by_role("button", name="Open paper help")).to_be_visible()

```